Computers & Security 148 (2025) 104098

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Survey of source code vulnerability analysis based on deep learning

Chen Liang, Qiang Wei, Jiang Du, Yisen Wang *, Zirui Jiang

Information Engineering University, Zhengzhou 450001, China

ARTICLE INFO

Keywords:
Vulnerability detection
Deep Learning
Similarity analysis
Code representation
Cyber security

ABSTRACT

Amidst the rapid development of the software industry and the burgeoning open-source culture, vulnerability detection within the software security domain has emerged as an ever-expanding area of focus. In recent years, the rapid advancement of artificial intelligence, particularly the notable progress in deep learning for pattern recognition and natural language processing, has catalyzed a surge in research endeavors exploring the integration of deep learning for the enhancement of vulnerability detection techniques. In this paper, we investigate contemporary deep learning-based source code analysis methods, with a concentrated emphasis on those pertaining to static code vulnerability detection. We categorize these methods based on various representations of source code employed during the preprocessing stage, including token-based and graph-based representations of source code, and further subdivided based on the types of deep learning algorithms or graph representations employed. We summarize the basic processes of model training and vulnerability detection under these different representation formats. Furthermore, we explore the limitations inherent in current approaches and provide insights into future trends and challenges for research in this field.

1. Introduction

Software vulnerabilities significantly compromise computer systems and IT infrastructure security. For instance, in 2017 the EternalBlue vulnerability exploited by the WannaCry ransomware affected computer systems in over 150 countries with estimated losses surpassing 4 billion dollars. Moreover, the development of the open-source community has led developers to frequently reuse open-source code in software development to reduce costs. However, this extensive reuse also facilitates the widespread spread of vulnerabilities, introducing latent risks to software security (Plate et al., 2015). Hence, early detection of vulnerabilities serves to mitigate the risk of system compromise and reduce the potential losses incurred by organizations or companies due to attacks exploiting these vulnerabilities (Chowdhury and Zulkernine, 2011). However, vulnerability analysis is a costly and critical process, demanding more from cybersecurity professionals and requiring advanced technologies for early detection.

Currently, software code vulnerability detection methodologies include static, dynamic, and hybrid analysis.

Static analysis for source code encompasses code similarity detection (Jang et al., 2012; Kim et al., 2017; Li et al., 2017; Sajnani et al., 2016; Wang et al., 2018), rule-based analysis (Lee et al., 2006), and symbolic execution (Li et al., 2013). Static analysis typically uses abstract interpretation to analyze program attributes such as variable value ranges, function call relationships, and control flows. However, static methods are unable to detect vulnerabilities that only emerge during program execution, possibly due to specific inputs or runtime environments.

Dynamic analysis methods, such as taint analysis (Karim et al., 2018) and fuzz testing (Gan et al., 2022; Pham et al., 2020), execute a program with specific inputs to monitor its runtime behaviors. Dynamic analysis requires specific input data as test cases to simulate the program's operation under various inputs. Nonetheless, these methods often suffer from lower code coverage and may require substantial time and resources for processing all inputs, especially in large and complex software.

Hybrid analysis combines static and dynamic approaches to leverage their strengths, enhancing analysis comprehensiveness and depth while mitigating their limitations (Alhuzali et al., 2018). For instance, hybrid analysis might use static analysis to identify potential defect areas in the code, followed by dynamic analysis to verify the presence of vulnerabilities under various runtime conditions.

However, all three techniques rely on a limited set of patterns based on known vulnerabilities (Croft et al., 2023). Static analysis relies on expert-defined feature sets, security rules, or execution paths; dynamic analysis on existing vulnerability test data and experience; and hybrid analysis on static rules to predict code areas that may contain

* Corresponding author.
E-mail addresses: liangc_hubugui@126.com (C. Liang), funnywei@163.com (Q. Wei), dujiang1101@126.com (J. Du), 851067568@qq.com (Y. Wang), 1419026412@qq.com (Z. Jiang).

https://doi.org/10.1016/j.cose.2024.104098
Received 8 April 2024; Received in revised form 7 August 2024; Accepted 2 September 2024
Available online 3 September 2024
0167-4048/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

vulnerabilities. Despite the introduction of various vulnerability detection methods in recent years, the rising number of vulnerabilities and reported CVEs still presents a significant challenge.

Scholars have extensively explored various vulnerability detection methods over the years. With the advancement of Machine Learning (ML), scholars have sought opportunities to apply various ML techniques to vulnerability detection. Theoretically, static analysis can identify all potential vulnerabilities across the source code. The "completeness" aligns well with ML's capability to identify patterns in large datasets. Consequently, ML-based static analysis methods for vulnerability detection have received extensive research attention.

ML-based vulnerability detection is a data-driven quality assessment process (Croft et al., 2023), primarily used at the source code level. This process views source code vulnerability detection as a binary classification task, aiming to classify source code into "vulnerable" or "non-vulnerable" categories by leveraging vulnerability knowledge.

Initially, research relied on expert experience to construct and use vulnerability-related features for labeling potentially vulnerable code. These features were then analyzed using ML algorithms. However, the diversity of vulnerability code patterns makes relying solely on expert-constructed features to express all characteristics impractical.

Advancements in ML, especially Deep Learning (DL) models, have significantly improved ML-based vulnerability detection's effectiveness. To further enhance detection accuracy and alleviate the burden on human experts in feature construction, numerous research has explored the potential of deep neural networks in automated vulnerability detection applications. Despite these endeavors, these methods still face many challenges.

This article aims to summarize recent advances in DL-based source code vulnerability analysis methods and outline technological trends. Additionally, it also discusses the current challenges and potential future directions in this field.

2. Background

This section provides a comprehensive background for the studies of software vulnerability detection using ML. It begins by establishing a clear and precise definition of software vulnerabilities, drawing upon authoritative sources and reconciling various perspectives in the cyber security field. The section then progresses to explore the landscape of static vulnerability detection methods based on ML, categorizing them into three main approaches: those based on software metrics, vulnerable code patterns, and anomalous behavior patterns.

2.1. Vulnerability definition

The concept of a software security vulnerability is a widely discussed topic in the field of software research. Ivan Krsul defined software security vulnerability in his 1998 paper (Krsul, 1998) as: An instance of an error in the specification, development, or configuration of software such that its execution can violate the security policy. Subsequently, Ozment refined Krsul's definition slightly, suggesting: A software vulnerability is an instance of a mistake in the specification, development, or configuration of software such that its execution can violate the explicit or implicit security policy. Ozment (2007) Ozment changed the term for the vulnerability from 'error' to 'mistake' and referenced the IEEE Software Engineering Vocabulary(IEEE Standard Glossary of Software Engineering Terminology, 1990) for further clarification.

To clarify the distinctions between these terms and select an accurate definition for software vulnerabilities, we refer to the IEEE Software Engineering Terminology Standard (Radatz et al., 1990), synthesizing four key terms: mistake, fault, error, and failure. According to IEEE Standards, a mistake is defined as: a human action that produces an incorrect result. A fault is defined as: an incorrect step, process, or data definition in a computer program. And faults are also known as flaws or bugs. Error is defined as: the difference between a computed, observed, or measured value or condition and the true, specified, or theoretically correct value or condition. A failure is: the inability of a system or component to perform its required functions within specified performance requirements.

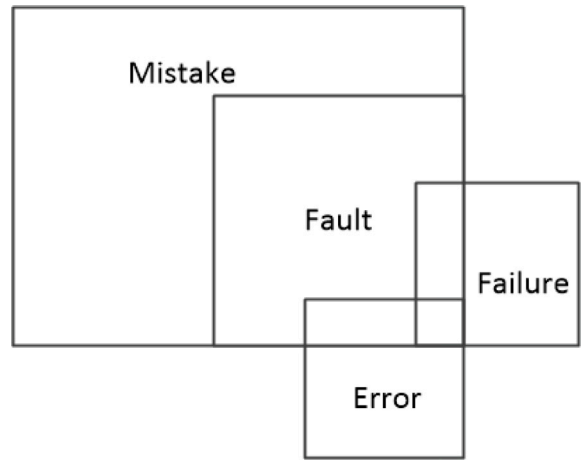


Fig. 1. The venn figure between mistake, fault, failure and error.

More precisely, a software vulnerability should be defined as a type of fault (fault, i.e., flaw or bug): A software vulnerability is an instance of a flaw that arises from mistake in the design, development, or configuration of software. This flaw may be exploitable by potential attackers, thereby violating established explicit or implicit security policies. The fundamental cause of software vulnerabilities is human mistakes, manifesting as faults. Executed vulnerable code segments do not necessarily violate security policies. It is only when data flows under specific or special conditions reach the flawed code that the execution of these code segments might contravene the security policies, result in an error, and ultimately lead to a failure. The relationships between these four concepts can be illustrated as follows in a Venn Fig. 1.

2.2. Source code vulnerability static detection method based on ML

In computer security, extensive research has focused on data-driven static vulnerability detection using ML. These methods emulate the process security practitioners undergo in searching for vulnerabilities, including preprocessing the source code, and inputting it into ML models to train the models' ability to distinguish between vulnerable and non-vulnerable code modules. According to their focus and implementation mechanisms, we categorize ML-based source code vulnerability detection methods into three main types.

2.2.1. Methods based on software metrics

Software metrics evaluate software quality using numerical indicators such as lines of code, Cyclomatic Complexity (Doyle and Walden, 2011), McCabe metrics (Mccabe, 1976), and Code Churn measurements (Nagappan and Ball, 2005). Many models that use software metrics assume complex code is harder to understand, maintain, and test, making it more vulnerable. Similarly, code that is frequently modified is more prone to errors, and thus, more likely to contain vulnerabilities. Although long used to measure software quality and predict defects, these metrics do not directly relate to code security.

In fact, vulnerable code blocks are not always complex or large. Therefore, ML models only based solely on these metrics often perform modestly in vulnerability detection. From the perspective of code audits, factors such as code complexity can be further considered but are usually not the decisive factor in determining the presence of vulnerabilities. Therefore, a refined feature set is necessary to accurately

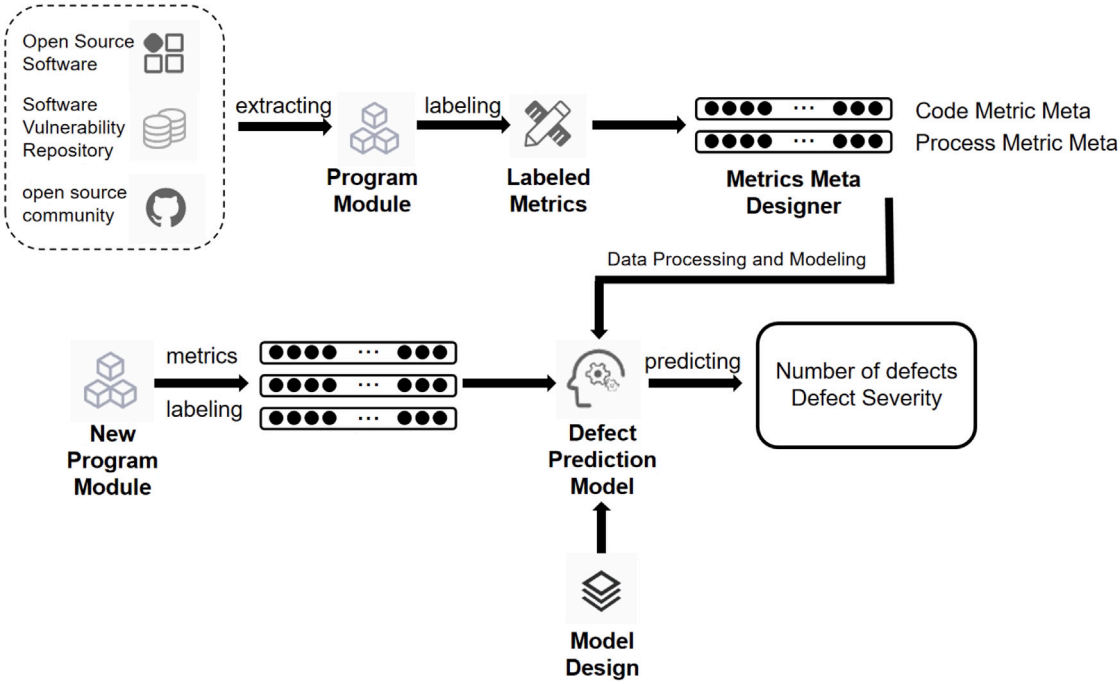


Fig. 2. The process of methods based on software metrics.

describe vulnerable code and train efficient classifiers for vulnerability discovery. The process of methods based on software metrics is illustrated in the Fig. 2.

Some research uses text mining from natural language processing to analyze source code and identify patterns by analyzing token frequency. For example, the Bag of Words (BOW) model turns source code tokens into vector representations for use as input features in classification algorithms (Scandariato et al., 2014). Additionally, some studies employ the N-gram method to extract code patterns and explore token relationships (Pang et al., 2015). However, these frequency-based text mining techniques cannot capture the deep semantic dependencies of software source code, only smoothing the co-occurrence relations. Scholars (Wang et al., 2016) also note that two Java files with identical tokens and frequencies can have vastly different semantics.

2.2.2. Methods based on vulnerable code patterns

Subsequent research pivots towards utilizing static code analysis tools to extract more complex semantic and structural features from datasets containing vulnerable source code.

Methods based on vulnerable code patterns rely on the understanding that code within a vulnerability might exhibit characteristics indicative of vulnerabilities. Although relying solely on security experts for feature definition may not precisely identify vulnerabilities. If a high-quality, large-scale labeled vulnerability dataset is available, and translated into representations understandable by ML mode. Models can identify features corresponding to vulnerable code patterns, which help vulnerability detection for unlabeled code.

Consequently, methods based on vulnerable code patterns often employ supervised learning techniques. These methods are focused on identifying code patterns that harbor security risks. Also, being data-driven methods, the cornerstone is a high-quality vulnerability dataset. The dataset now primarily derived from existing code security audits, records of fixed vulnerabilities (Chakraborty et al., 2022; Fan et al., 2020; Gkortzis et al., 2018; Zheng et al., 2021b; Zhou et al., 2019), and synthetically generated vulnerable code based on artificial vulnerability patterns (Black, 2018). Based on the dataset, semantic and structural features are extracted using static analysis tools. These features are drawn from different forms of program representations produced by static analysis, including token sequences, Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), Data Flow Graphs (DFGs), and Program Dependency Graphs (PDGs). Yamaguchi et al. (2014) proposed method amalgamates AST, CFG, and PDG into a unified representation termed the Code Property Graphs (CPGs), offering a graph structure with enhanced dimensionality for describing code properties.

Methods typically based on vulnerable code patterns encompass several steps, including data collecting, data labeling, preprocessing (feature engineering), model design, model validation and testing, as illustrated in the Fig. 3.

Compared to simple software metrics and frequency-based code features, features extracted from the output of code analysis tools reveal richer code information. Each form of program representation examines the source code from a specific perspective. For example, ASTs represent the code structure in a tree view; CFGs, DFGs, and PDGs analyze how variables flow through the code.

2.2.3. Methods based on anomalous behavior patterns

Furthermore, scholars suggest that vulnerable code often exhibits anomalous behaviors, distinct from normal patterns. Such anomalies may originate from the code's execution paths, unusual use cases of the code, or abnormal patterns compared to conventional code repositories. Methods based on anomalous behavior patterns aim to discover potentially vulnerable anomalous behaviors in code and strive to uncover the underlying causes of vulnerabilities rather than merely identifying vulnerabilities. In contrast to vulnerable code pattern methods, anomalous behavior methods typically use semi-supervised or unsupervised learning to detect anomalies by analyzing low-frequency events in model-generated error or probability distributions.

Additionally, anomalous behavior methods rely more on normal code data to establish a baseline for normal behavior, thus requiring lower demands on data set collection and construction. During the detection process, the model aims to identify behaviors deviating from the established normal behavior patterns. It means that anomalous patterns do not need to be predefined.

Early studies used security practitioners' experience to create rules as templates for identifying potential errors or vulnerabilities in non-compliant code. Later research focused on features from abnormal API

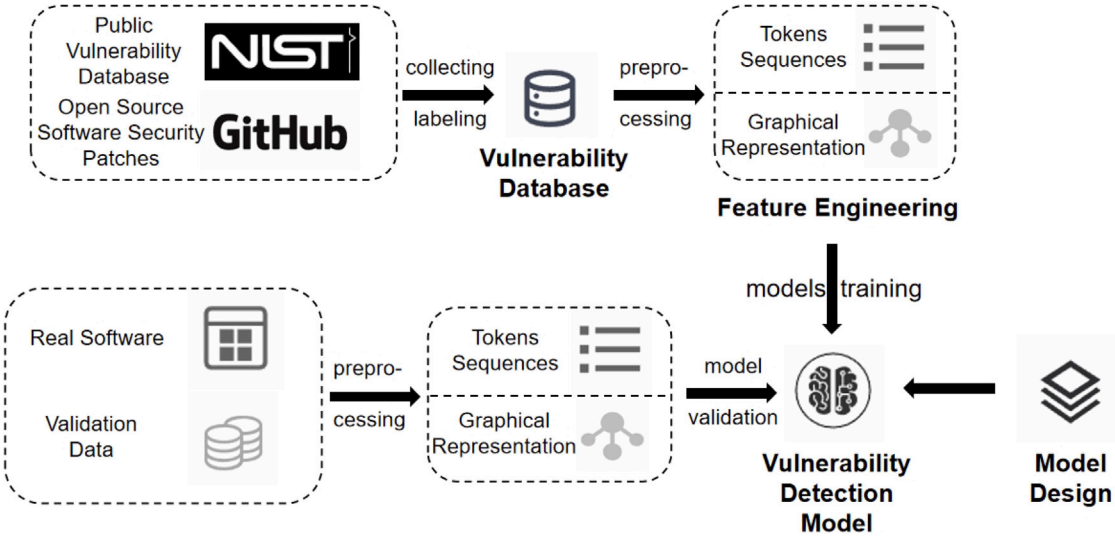


Fig. 3. The process of methods based on vulnerable code patterns.

use (Acharya et al., 100), library imports and function calls (Neuhaus et al., 2007), and improper API checks (Yamaguchi et al., 2013) to detect vulnerabilities. However, these features are typically associated with specific types of vulnerabilities. Using rules or features from anomalous code patterns as standards risks generating many false positives. Overall, methods based on anomalous behavior patterns are better suited for detecting specific vulnerabilities.

This article synthesizes the collection and summary of DL-based vulnerability detection methods in recent years. At present, Static vulnerability detection methods based on vulnerable code patterns become mainstream within this field. According to the representations of source code, methods based on vulnerable code patterns can be further divided into those based on token sequences representations and those based on graph representations. The following Section 3 and Section 4 will analyze the relevant research on these two methods.

3. Vulnerability detection methods based on token sequences

Source code inherently possesses both lexical structures and word semantics. During the compilation process, source code undergoes steps such as lexical, syntax, and semantic analysis by the compiler, segregating different tokens from the continuous stream of characters. Tokens are the smallest syntactical unit of code, including keywords, operators, identifiers, literals, and so forth. From the compiler's perspective, code is a sequence of tokens bearing specific meanings. Therefore, extracting lexical relationships and semantic information from source code via specific rules to form token sequences represents a reasonable abstraction of the source code. Token sequences are also conducive to vector embedding, providing input for ML models. Hence, transforming source code into token sequences is one of the most common and classic preprocessing methods in the ML-based static vulnerability detection domain.

The generic basic operations of vulnerability detection methods based on token sequences are shown in the Fig. 4 and mainly include two phases: model training and detection. During the training phase, open-source and vulnerable code serve as input data and are sliced according to points of interest (POIs) to generate slice sequences. The slicing sequences are then labeled using vulnerability detection tools based on their origin, obtaining sequences with security or vulnerability labels. Subsequently, the code sequences are standardized by removing non-ASCII characters, comments, and converting variables and function names into symbolic names. Finally, sequences are vectorized by various algorithms in preparation for being input into ML or DL models. After designing and constructing various ML models and conducting model training, a vulnerability detection model is obtained. During the detection phase, the code to be examined follows similar slicing, labeling, standardization, and vectorization processes as in the training phase. The vectorized code is then fed into the vulnerability detection model for binary classification to determine the presence of vulnerabilities and output the security detection results for the code in question.

This section classifies and outlines methods that use token sequences to represent source code according to different ML models.

3.1. Methods based on recurrent neural network (RNN) models and their variants

We examined six seminal papers utilizing neural network models, including Long Short-Term Memory networks (LSTM) and Gated Recurrent Unit networks (GRU), for code feature extraction. These studies consistently achieved static vulnerability detection through the analysis of token sequence.

Addressing traditional vulnerability detection's reliance on expert knowledge and high false-positive rates, (Li et al., 2018) introduced VulDeePecker. Its core component, the code gadget, consists of code snippets extracted using heuristic methods to encapsulate the semantic and structural essence of vulnerability data streams. The system selects elements such as API calls, arrays, and pointer variables, converting vulnerable code into a related collection, i.e., a code gadget. Authors select vulnerability code from the CWE database, specifically CWE339 and CWE119, convert these into code gadgets, and encode them as feature vectors via lexical analysis and Word2Vec for bidirectional LSTM (BLSTM) training. Results show that VulDeePecker reduces false positives and identifies four new vulnerabilities in real-world software, underscoring its practical value.

Li et al. (2019) presented LAVDNN, an efficient auxiliary vulnerability detection approach. LAV- DNN extracts canonically named function names as significant semantic features to capture the function's features, suggesting these encapsulate both functionality and vulnerability potential. In practice, LAVDNN extracts function names from open-source programs and vectorizes these at the character level using one-hot encoding to derive semantic features. These features serve as input for a BLSTM for model training. The training set includes sensitive function names from CVE and non-sensitive names from open-source software. Results indicate that LAVDNN achieved precise categorization, with fewer false positives compared to other tools in tests with FFmpeg and LibTIFF.

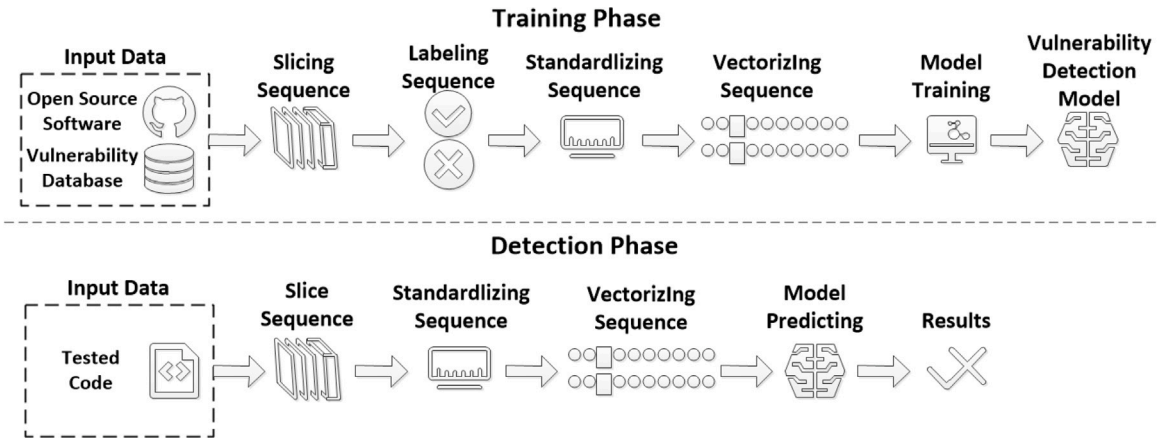


Fig. 4. The process of methods based on token sequences.

The multi-type vulnerability detection system μVulDee- Pecker (Zou et al., 2019) is an enhancement of VulDeePecker, capable of simultaneously detecting the presence of code vulnerabilities and determining their types. μVulDeePecker considers data and control dependencies when forming code gadgets, introducing code attention mechanisms and a new building-block BLSTM model to augment feature fusion. μVulDeePecker initiates by generating dependency graphs with the joern tool and amalgamating these with function call relationships to construct system dependency graphs. It then identifies pivotal nodes for code gadgets and applies predefined syntactic rules for code attention. After vectorization, code gadgets and code attention are input into three BLSTM to extract, combine, and fuse global and local features, culminating in precise vulnerability classification by the classifier. μVulDeePecker utilizes vulnerability data from SARD and NVD, covering 116 types of CWE vulnerabilities. Experiments show its performance surpassing the original VulDeePecker.

Li et al. (2022b) introduced the SySeVR system, which efficiently vectorizes program source code by incorporating both the syntactic and semantic information of vulnerabilities. SySeVR defines the concepts of Syntax-based Vulnerability Candidates (SyVCs) and Semantics-based Vulnerability Candidates (SeVCs) initially. SyVCs are extracted using the Checkmarx tool, and then expanded into SeVCs through Joern program slicing, which includes the syntactic information from SyVCs as well as the semantic information of data and control dependencies extracted during the expansion. SeVCs are then encoded into fixed-length vectors using Word2Vec, which are fed into various deep neural networks for training. Experiments utilizing data from NVD and SARD demonstrate that bidirectional GRU (BGRU) achieves the best performance among several neural networks. In practical applications, the SySeVR system, combined with the BGRU model, discovered 15 new vulnerabilities in four software programs that were not reported in NVD, verifying its practicality and potential.

Garg et al. (2022) suggested that vulnerabilities in software programs are often discovered over time, with the majority being identified later than the point of introduction. This temporal discrepancy can result in misclassifying vulnerable code as safe during model training, thus learning incorrect features. The authors develop TROVON, a method for identifying potential vulnerabilities by analyzing the historical code of known vulnerabilities and their patched versions. TROVON leverages publicly available vulnerability records from NVD, extracts function-level code snippets, and conducts pair analysis of historical versions. It uses defined rules to label code, determining the modified code as potentially vulnerable. After cleaning irrelevant information, standardized code is used to train and test LSTM models to predict code susceptibility to attacks. Evaluation shows that TROVON's prediction accuracy surpasses other vulnerability prediction methods.

Li et al. (2022a) introduced the VulDeeLocator model for vulnerability detection and localization in C language, aiming to address the issue of inaccurate capture of control flow and define-use information across multiple files. The model links define-use relationships across files, substitutes source code representations with intermediate code, and employs an extended Bidirectional RNN (BRNN-vdl) for precise vulnerability identification and localization. VulDeeLocator selects source code snippets with vulnerability syntactic features, generates IR code and its slices using clang and dg tools, creating intermediate code and Semantics-based Vulnerability Candidates (iSeVCs). It then labels the vulnerability status and locations of iSeVCs, standardizes and vectorizes them for training in BRNN-vdl. With additional layers for refinement and attention focusing, BRNN-vdl outputs the locations of vulnerable code lines. Experiments using NVD and SARD data and LLVM intermediate code show VulDeeLocator's effectiveness on both synthetic and real datasets.

Wartschinski et al. (2022) proposed Vudenc (Vulnerability Detection with DL on a Natural Code-base), a vulnerability detection method for Python language. Vudenc embeds source code using Word2Vec. Based on the assumption of fundamental similarities between programming and natural languages, Vudenc uses LSTM to classify token sequences in vulnerable code at a fine-grained level, identifying potential vulnerability areas and providing confidence levels. Additionally, the authors gathered 1009 security code changes from different GitHub repositories for model training. Results indicate that Vudenc achieves higher recall, precision, and F1 scores.

3.2. Methods on convolutional neural network (CNN) models and their variants

This subsection examines two seminal papers utilizing CNN for code feature extraction. These works successfully achieve efficient static vulnerability detection based on sequences of tokens.

Russell et al. (2018) developed a machine-learning-based vulnerability detection system based on a vast dataset of C/C++ open-source code, creating a large open-source vulnerability dataset. During dataset construction, a C/C++ lexical analyzer simplifies the vocabulary and extracts key semantics, standardizing the code to facilitate transfer learning and prevent overfitting, with the vocabulary limited to 156 keywords. Redundancy reduction enhances the detection efficiency of new code, retaining 10.8% of functions. Lastly, static analysis tools such as Clang, Cppcheck, and Flawfinder filter results and map to the CWE vulnerability database for accurate vulnerability labels. To evaluate dataset quality and ML-based detection methods, the authors trained different models on the constructed dataset and compared the results, showing that CNN models achieved the best performance.

Zhiquan et al. (2022) introduced SEVULDET, a semantic enhancement based vulnerability detection method. It points out that program slicing should include related statements and maintain complete logic.

Table 1
Comparison between RNNs with their variants and CNNs.

Features	RNNS with their variants	CNNs
Sequence Data Processing	Efficiently captures inter-token dependencies, suitable for arbitrary input lengths	Parallel processing for efficient extraction of local features
Contextual semantic understanding	LSTM/GRU utilize gating mechanisms to capture forward dependencies, BLSTM/BGRU can capture bidirectional dependencies	Advantageous for extracting short-range dependences but lack in capturing long-range dependencies
Structured Information Understanding	Proficient in handling structured information within sequences, facilitating the capture of semantic and structural features	Exhibits strong local correlations but not suitable for complex structured code features
Computational Cost	High computational cost and low training efficiency (especially BLSTM/BGRU), challenging for parallel processing	Strong parallel processing capabilities with relatively high efficiency
Gradient Issues	Faces problems of gradient vanishing and explosion	Do not involve temporal issues, thus no such problems
Long-range Dependencies	Difficulties in processing long sequences, hard to capture long-range dependencies	Not adept at capturing long-range dependency structures
Sequential Processing Capability	Strong sequential processing capability	Lack in sequential processing capability, unsuitable for temporal data

But conventional slicing methods often overlook the context between statements, lacking appropriate context paths and remaining dependency details, leading to semantic loss. SEVULDET adopts a path-sensitive slicing method, extracting richer path and control flow logic. It incorporates spatial pyramid pooling layers in CNN and adds multi-layer attention mechanisms to handle semantically variable-length gadgets, avoiding semantic loss from traditional slicing methods' truncation or padding operations. Evaluations on SARD and NVD datasets indicate that SEVULDET surpasses conventional methods, raising the F1 score to 94.5% and discovering unreported new vulnerabilities in the Xen software.

3.3. Summary

Representations based on token sequences more closely approximate the semantic information of source code than other types of representations do. However, source code differs from natural language in that it embodies structural information, particularly control flow and data flow, which are crucial for a comprehensive understanding of code semantics. The emergence and triggering of vulnerabilities are influenced by both semantic and structural information. Hence, in DL-based vulnerability detection methods that symbolize source code as token sequences, the data preprocess is pivotal. The capability of preprocess phase to capture the complete semantic and structural information of the code affects the effectiveness of the final vulnerability detection methods.

Some preprocess methods treat source code similarly to natural language, using tokenization techniques for code segmentation and normalization, followed by the creation of token sequences. Such methods are unable to preserve the structural information of the source code, which results in lower performance in vulnerability detection. Subsequent methods propose using various tools or methodologies for preprocessing source code and generating sequences, thereby reinforcing the representation of structural information, especially control flow and data flow.

Following the finalization of code preprocessing and token sequence generation, ML models are employed to analyze the token sequences, endeavoring to abstract and identify high-dimensional vulnerability features. Consequently, the choice of different ML models, especially those DL models with diverse principles and architectures, is also critical for vulnerability detection performance. Currently, token sequence-based methods predominantly employ RNN models, especially the BLSTM and BGRU, which are frequently used for sequence feature extraction. The Table 1 contrasts the advantages and disadvantages of using RNN and its variants for token sequence-based vulnerability detection methods.

Table 2 summarizes recent methods that represent source code with token sequences, including the models used, principles, encoding methods, and vulnerability datasets employed.

4. Vulnerability detection methods based on graphs

In contrast to methods based on token sequence representation, graph-based representations can illustrate more complex code information with greater nuance. Current methods proposed encompass representations based on Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), and Program Dependence Graphs (PDGs). Both tree structures and the more intricate PDG and CFG represent nodes with tokens while maintaining explicit connections and hierarchical divisions between nodes. This enables a more precise portrayal of the complex syntactical relationships and the semantic flow of calls. Moreover, vulnerability detection methods that utilize different graph representations employ distinct processing approaches and training detection processes, reflecting the varied emphases of these graphs on different code aspects. We endeavor to outline the common fundamental operational steps of methods based on graph representations of source code, as illustrated in Fig. 5.

During the training phase, methods based on graph representations generate graph representations of input data using different tools at the function level. And existing research (Zou et al., 2024) indicates that representations at this level contain a lot of noise (the code unrelated to the vulnerability), which can impact the efficiency of DL models in extracting vulnerability features. Therefore, after obtaining the initial representations, scholars typically streamline these by removing structures unrelated to the vulnerability semantics or adding information potentially relevant to these semantics. We still describe this step as graph slicing. After obtaining the graph slices, the graph is labeled, and nodes' tokens are vectorized according to different algorithms for model training, ultimately obtaining the vulnerability detection model. In the detection phase, the code undergoes similar processes of graph representation, slicing, and node vectorization, which are then fed into the vulnerability detection model for graph-based binary classification. Based on different graph representations, this section will categorize and outline methods of using graph representations of source code.

4.1. Methods based on CPG representation

We review four influential papers on vulnerability detection that utilize CPG representations. These papers all implement Graph Neural Network (GNN) for embedding and feature extraction, achieving vulnerability detection based on CPG.

Table 2
Methods based on token sequences.

Number	Literature	Year	DL method	Principles	Encoding method	Dataset
1	VulDeePecker	2018	BLSTM	POI Slice	Word2Vec	–
2	LAVDNN	2019	BLSTM	Function names as semantic Features	Word2Vec	–
3	μVulDeePecker	2021	BLSTM	Slice(PDG,SDG,POI)(data+control dependencies)	Word2Vec	SARD+NVD
4	SySeVr	2022	BGRU	Checkmarx grammatical features+Slicesemantic features	Word2Vec	SARD+NVD
5	TROVON	2022	LSTM	Based on vulnerability patches	–	NVD
6	VulDeeLocator	2022	BRNN	Slice+clang IR	Word2Vec	SARD+NVD
7	VUDENC	2022	LSTM	Tokenizer participle	Word2Vec	–
8	Russel et al.	2018	CNN	Lexical analyzer	–	–
9	SEVulDet	2023	CNN	Path sensitive slice+multi-layer attention	Word2Vec	SARD+NVD

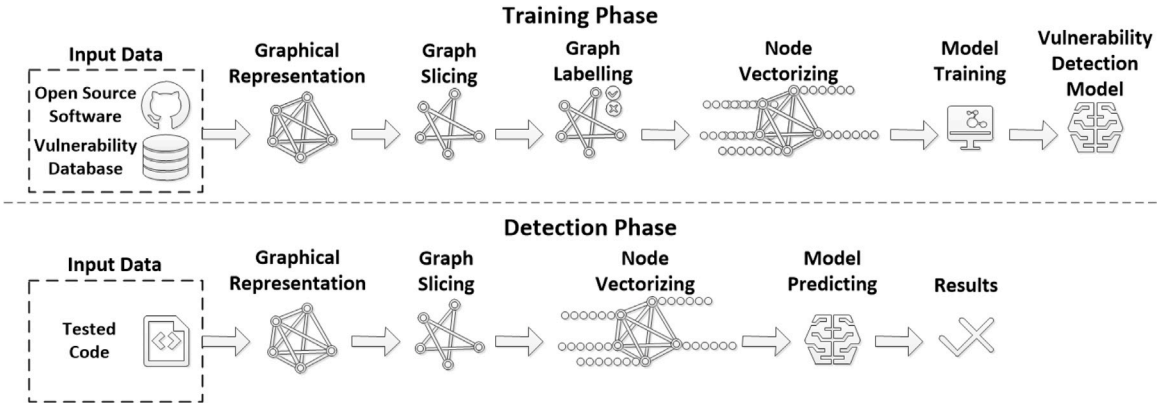


Fig. 5. The process of methods based on graphs.

In the study (Xiaomeng et al., 2018), the authors proposed CPGVA, a DL-based vulnerability detection method. CPGVA transforms source code into a CPG and stores it in a database with the Joern tool. After encoding CPG nodes with Word2Vec, CPGs are fed into an LSTM model for training. Experiments demonstrate that CPGVA trained on the SARD dataset improves performance in various aspects.

Duan et al. (2019) introduced VulSniper, a method for fine-grained vulnerability detection utilizing attention mechanisms. VulSniper employs the CPGs, adopting a dual structure from bottom-up and top-down to capture key features of different parts of the code. Meanwhile, VulSniper designs a 144-dimensional vector to describe vulnerability features. The neural network used by VulSniper learns feature weights through embedding and attention modules and ultimately performs binary classification through a fully connected layer. In experiments on the SARD dataset, VulSniper exhibits higher F1 scores than existing methods.

Ding et al. (2022) introduced VELVET, a vulnerability detection method at the code line level using ensemble learning. VELVET integrates two neural network models, the Gated GNN(GGNN) and Transformer, to identify various vulnerability patterns. VELVET utilizes the JULIET (synthetic) and D2 A (real projects) datasets for experimentation. Given the smaller size of the D2 A dataset, VELVET undergoes pre-training with a larger learning rate on JULIET before fine-tuning on D2 A with a smaller learning rate. During training, source code is converted into CPG, and nodes are vectorized using Word2Vec. Subsequently, both GGNN and Transformer models are trained separately – the former learning local sequence semantics and the latter global semantics. During the prediction phase, results from both models are integrated to produce the final prediction. The experiment demonstrates that VELVET's performance surpasses that of existing static analysis methods by a factor of 4.5.

Li et al. (2023) addressed the issue of uneven distribution of vulnerabilities across software projects by proposing VulGDA, a cross-domain vulnerability detection method based on DL. In the data preprocessing stage, VulGDA conducts lexical analysis, syntactic analysis, and AST construction of the source code, then creates a CPG graph based on the AST. Node embeddings are generated using a pre-trained word embedding model, and then a GGNN is used to obtain embedding vectors by propagating and aggregating neighbor information defined in the CPG. To achieve cross-domain detection, VulGDA employs transfer learning and designs a feature generator based on deep domain adaptation, creating discriminative features for vulnerability detection that are stable across domain shifts, thereby enhancing cross-domain detection performance. Experiments on the Devign and Reveal datasets show that VulGDA's cross-domain detection performance surpasses existing methods.

4.2. Methods based on PDG representation

This subsection discusses five vulnerability detection methodologies using Program PDG representations. These methods exploit PDG's complex semantics and structural information to enhance vulnerability feature extraction through GNN, consistently achieving high vulnerability detection levels.

Li et al. (2021) introduced IVDETECT, an interpretable method for vulnerability detection. IVDETECT applies a vulnerability detection model to identify and categorize vulnerabilities, followed by an explanation model to pinpoint the most pertinent code statements associated with the vulnerabilities. IVDETECT begins by producing a sub-token sequence of the code, and then extracts ASTs of functions, variable names and type features, as well as data and control dependency characteristics. Subsequently, it employs a BGRU enhanced by the Attention mechanism to integrate all five feature types to achieve a comprehensive code representation. The representation in conjunction with the PDG is then processed by the Feature-Attention Graph Convolutional Network (GCN) model for training. Moreover, IVDETECT utilizes the GNNExplainer model to elucidate classification results. GNNExplainer identifies key subgraphs or sub-features within the PDG as explanatory outcomes for the classification. The tests on Bigvul, Reveal, and Devign datasets demonstrate high performance, with the GNNExplainer showcasing superior accuracy in interpretation.

Wu et al. (2022) presented VulCNN, a vulnerability detection method grounded in DL, capable of efficiently transforming source

code into images while preserving program details and achieving scalability and precision in vulnerability scans of large-scale source code. VulCNN constructs a PDG via program analysis, appends graph structural information to lines of code through centrality analysis and represents centrality results within a three-dimensional image space. After, it utilizes a CNN model for vulnerability detection. VulCNN also employs deep visualization techniques to generate heatmaps which assist security analysts in locating and understanding vulnerabilities. Tests on a dataset comprising 13,687 vulnerable and 26,970 non-vulnerable functions shows that VulCNN's accuracy surpasses that of eight other vulnerability detection methods. Furthermore, investigation into over 25 million lines of real software code leads to the discovery of 73 previously unreported vulnerabilities not recorded in the NVD.

Zou et al. (2024) introduced mVulPreter, a method merging the advantages of slice-level and function-level methods. The authors note that slice-level detection can precisely identify the problematic code causing vulnerabilities, while it fails to provide a comprehensive view of the vulnerabilities. Conversely, function-level methods deliver a global perspective with the caveat of including noisy code segments. The mVulPreter combines the comprehensiveness of function-level with the precision of slice-level detection. The mVulPreter initially preprocesses the data and utilizes Joern to extract PDGs, then slices the data based on defined sensitive points. Concurrently, it labels the slices and embeds them using Sen2Vec and GGNN, resulting in fixed-length vector inputs for model training. The DL model ingests code slices and outputs vulnerability probability scores. Additionally, mVulPreter trains an attention-based GNN model (GGNN + Attention Mechanism) at the function-level, which predicts the vulnerability of test functions and the attention weights of slices within these functions. Ultimately, mVulPreter combines the outcomes of both models to produce a significance score for lines of code. Utilizing a subset of the Big-vul dataset for training, mVulPreter successfully identified numerous unreported vulnerabilities in software such as Libav, confirming its effectiveness.

Mirsky et al. (2023) introduced VulChecker, a vulnerability detection method capable of pinpointing the location of vulnerabilities in code at both the instruction and source code levels and analyzing the types of vulnerabilities present. The authors posit that semantic structure graphs should be captured at the instruction level in order to enhance the capability of DL-based vulnerability detection methods in reasoning and recognizing vulnerabilities, rather than at the source code level. Therefore, VulChecker begins by compiling the source code using a custom LLVM compiler toolchain, generating representations optimized for GNN, namely the enhanced PDG (ePDG). Nodes of the ePDG correspond to elementary machine-level instructions, while edges represent the control flow and data dependencies between these instructions. VulChecker also designs a GNN-based vulnerability detection model via a message-passing mechanism, leveraging intricate code semantics to identify potential vulnerabilities. In addition, the authors introduce a data augmentation technique that combines synthetically generated vulnerable code with real-world code to create more authentic training samples. In experiments, VulChecker augments real project ePDG from GitHub by inserting ePDG subgraphs of artificially synthesized vulnerabilities from the Juliet. Comparative tests demonstrate that VulChecker is capable of providing security analysts with vulnerability predictions that minimize both false positives and false negatives.

Hin et al. (2022) proposed LineVD, a DL-based line-level vulnerability detection method. LineVD employs CodeBERT (Feng et al., 2020b) for code tokenization, extracts function-level and statement-level code features while concurrently learning PDGs through Graph Attention Networks (GAT). LineVD also leverages information propagation mechanisms to learn the graph-structured data, obtaining data and control dependencies of the code. Lastly, by integrating shared linear and dropout layers along with element-wise multiplication operations, LineVD trains a classifier for joint learning at both function and code line levels. Tests on the Big-vul dataset reveal that LineVD performs well in statement-level detection.

4.3. Methods based on other graphs and combined graph representation

Moreover, graphs such as CFGs, DFGs, and Call Control Graphs (CCGs) can also characterize various code features from control flow, data flow, and beyond. Related methods utilize these graph representations to train GNN for embedding and feature extraction, successfully applying them to the detection of vulnerabilities.

Viet Phan et al. (2017) introduced a method for vulnerabilities' stealthy characteristics. This method employs Program CFGs to depict the execution of programs, combined with Multi-View Multi-Layer Directed Graph Convolutional Neural Networks (DGCNN) for processing large-scale graph structures. Initially, the method compiles source code to obtain assembly instructions and generates CFGs from these instructions to learn vulnerabilities' semantic features. Experiments on four real-world datasets demonstrate that the performance of this method significantly surpasses other DL-based vulnerability detection methods.

Zhou et al. (2019) introduced Devign, a method that effectively identifies vulnerabilities through comprehensive semantic analysis. In the preprocessing stage, Devign transforms the source code into a composite code representation based mainly on ASTs and explicitly encodes different levels of program control and data dependencies as hybrid edges, resulting in a combined graph of AST, CFGs, and DFGs. Each node of the combined graph utilizes the code's Word2Vec encoding and type label encoding for representation. Subsequently, Devign employs a Gated Graph Recurrent Network (Gated GRN) to learn node embeddings and achieve graph-level classification. Gated GRN propagates information through different types of edges, updates states based on node interactions, and employs convolutional layers and dense networks to extract graph-level features for vulnerability prediction. Additionally, the authors collect marked vulnerability data from four C-language libraries to construct a dataset for evaluating Devign, which has been publicly released for further DL-based vulnerability detection methodologies (named the Devign dataset for ease of reference). Results indicate that Devign exceeds baseline methods in terms of accuracy and F1 scores, with an average accuracy increase of 10.51% and an F1 score improvement of 8.68%.

Hellendoorn et al. (2020) proposed two novel hybrid models to tackle the limitations of traditional graph-based vulnerability detection methods, which rely on graph-based message passing to represent code semantics, resulting in high costs and the inability to learn all semantic information in the graph. The first model, termed the "Graph Sandwich Model", wraps the traditional graph message passing layer within a newly introduced sequential message passing layer, allowing it to capture both local and global information within the graph. The second model, Graph Relational Embedding Attention Transformers (GREAT), extends Transformers' position embedding to depict structural relationships. These hybrid models balance global and structural characteristics, integrating the advantages of different approaches to achieve significant improvements in performance, training efficiency, and parameter count.

Cao et al. (2021) introduced BGNN4VD, a vulnerability detection method leveraging Bidirectional GNN (BGNN) and CNN. In the preprocess phase, BGNN4VD constructs a CCG that fuses AST, CFG, and DFG to capture syntactic and semantic relationships. The CCG comprises AST nodes linked by various edge types, with statement and predicate nodes interconnected via CFG and DFG. Thus, the types of edges in the CCG reflect the syntactic and semantic relationships between different nodes. BGNN4VD converts the CCG into vector input for the GNN, using backward edges to identify code features, then extracts features via the CNN and identifies vulnerabilities with classifiers. The authors build a real dataset containing 2,149 vulnerable functions for experimentation, showing that BGNN4VD achieves higher detection performance compared to traditional methods.

DeepWukong (Cheng et al., 2021) initially generates the CFG and Variable Flow Graph (VFG) of code and builds the PDG based on control and data flow. It then conducts forward and backward traversals across

POI on the PDG, forming and standardizing the program's Extended Flow Graph (XFG, a PDG subgraph). Each statement token in the source code (i.e., each node on the XFG) is converted into a vector representation using Doc2Vec. Subsequently, structured information from XFG edges and unstructured information from vectorized code tokens are used as inputs for the neural network. Experiments on the SARD dataset with many graph-neural networks, including GCN, GAT, and k-GNNs, are conducted and compared with existing methods. The results shows that DeepWukong with k-GNNs demonstrates superior performance.

Zheng et al. (2021a) defined a new code representation called the Slicing Property Graph (SPG), which aims to represent the semantic and structural information of code while eliminating as much irrelevant and redundant information as possible to reduce the complexity of the graph. Inspired by Li et al. (Li et al. 2022a), the SPG construction incorporates two additional Syntax-based Vulnerability Candidates (SyVC) to the original four, broadening the vulnerability coverage in program slices. Using SyVC as the slicing criterion, the authors leverage program slicing to extract potentially vulnerability-related slice nodes and then generates SPG by employing the extracted slice nodes' data dependencies, control dependencies, and function call dependencies as edges. Additionally, the authors introduce a new DL-based vulnerability detection method, VulSPG, which employs an improved GCN model to represent SPG with multiple attributes and dependencies, aided by a triple-attention mechanism to highlight vulnerability features. Experiments on the SARD and NVD datasets demonstrate the effectiveness and efficiency of VulSPG.

4.4. Methods based on AST representation

The methods that utilize ASTs representations slightly diverge from the three graph-based methods previously discussed. The graph-based methods undergo a comprehensive series of preprocessing and vectorization processes. These processes are designed to yield detailed graph representations, encompassing nodes and edges. The models predominantly employed for these methods are GNNs and their variants. In contrast, AST-based methods often involve representing the code as an AST and then converting the AST into a sequential representation through different traversal algorithms. This approach shares similar steps with token sequence-based representations, including token sequences, sequence normalization, and sequence vectorization. Consequently, RNN models and their variants are predominantly adopted in AST-based methods. An overview of the steps in AST-based vulnerability detection methods is as follows. In contrast, AST-based methods primarily focus on representing the code as an AST. Following this, they convert the AST into a token sequence via various traversal algorithms. This approach bears similarities to token sequence-based representations, which involve processes such as sequence normalization and sequence vectorization. As a result, RNN models and their variants are primarily adopted in AST-based methods. The steps involved in AST-based vulnerability detection methods can be summarized as follow Fig. 6.

Lin et al. (2017) introduced POSTER, a cross-project, function-level vulnerability detection tool. POSTER initially extracts the code's AST using CodeSensor and then implements a specialized BLSTM. The network's first layer is dedicated to embedding code sequences. This is followed by two stacked LSTM layers that learn high-level abstractions. The final layers employ ReLU and linear activation functions, simplifying the output to a one-dimensional format. Tested on over 6000 manually annotated functions, the results demonstrate a significant superiority of AST-based vulnerability detection methods over traditional code metric-based approaches.

Lin et al. (2018) developed a syntax analysis tool, ANTLR, to convert source code into AST. It then serialized the AST using a depth-first traversal approach to a fixed length, embedding the serialized AST into vectors with the Word2Vec method. This model, trained with LSTM, aims to learn high-level feature representations and apply them to train classifiers for vulnerability prediction. Experimental results suggest that this method exceeds traditional methods based on code metrics in terms of accuracy.

Wang et al. (2021) presented FUNDED, a flow-sensitive, AST-based vulnerability detection model. This model consists of a GNN-based vulnerability detection model and a framework for collecting vulnerable code. In the detection model, FUNDED employs ASTs to construct program graphs, encoding the connectivity of graph nodes into a program graph matrix. This matrix is subsequently fed into a GGNN model for learning and training. The vulnerable code collection framework utilizes a set of prediction models to independently determine whether code commits in open-source code bases are security-related. This framework identifies code changes associated with vulnerabilities for model training. In experimental comparisons, FUNDED outperforms six other models.

Feng et al. (2020a) introduced an AST processing method designed to transform source code into vector representations, meticulously preserving its syntactic attributes. This technique initiates by parsing the source code into an AST, subsequently extracting function nodes, and then traverses the AST to align user-defined names with fixed identifiers. The process culminates in converting the AST into a node sequence via a preorder traversal. Upon employing Word2Vec to project the node sequence into a vector representation suitable for BGRU training, this method further incorporates pack-padded techniques to obviate the necessity for data truncation or padding. Utilizing the SARD dataset for empirical validation, this method exhibits notable accuracy and minimal false-positive rates in processing voluminous datasets.

Liu et al. (2019) introduced DeepBalance, a novel method aimed at mitigating the challenge of dataset class imbalance in DL-based vulnerability detection, which adversely affects the precision of DL models. DeepBalance commences by transforming code into AST sequences and leveraging BLSTM for feature extraction, followed by the construction of a classification model utilizing the random forest algorithm. Furthermore, DeepBalance integrates fuzzy oversampling techniques to equilibrate the dataset via synthetic instances. Empirical evaluations on real-world datasets, encompassing projects such as LibTIFF, LibPNG, and FFmpeg, demonstrated that DeepBalance substantially enhances the efficacy of vulnerability detection.

Dam et al. (2021) proposed an LSTM-based code detection methodology that generates ASTs through lexical parsing and morphs them into code token sequences, thereby acquiring fixed-length vector representations for each code snippet. To facilitate cross-project detection, it introduces the concept of a "codebook" derived from computer vision, equating each token in a file to a salient point (analogous to the most informative pixel in images), and applies k-means clustering to establish a codebook of all significant points, thereby assigning the nearest cluster centroid to a token as its global feature during the clustering process. An evaluation of the predictive efficacy of local versus global features reveals that this technique outperforms existing methods reliant on software metrics and bag-of-words models in forecasting outcomes across 18 Android and Firefox applications, in both intra-project and cross-project detection scenarios.

Lin et al. (2021) highlight the scarcity of vulnerability data as a significant impediment — termed a cold start problem — to the advancement of DL-based vulnerability detection methodologies. In response, the authors introduce a LSTM-based vulnerability detection framework that extracts vulnerability features from a diverse array of data sources, including synthetic vulnerability datasets and authentic historical vulnerability data. This framework consists of dual BLSTM, individually trained on the SARD and historical vulnerability datasets. The process begins by transforming functions into ASTs, followed by a depth-first traversal to compile sequence data, upon which features are elicited using BLSTMs. Finally, combining both sets of features with a random forest classifier for vulnerability detection, the method proves to perform better than existing approaches.

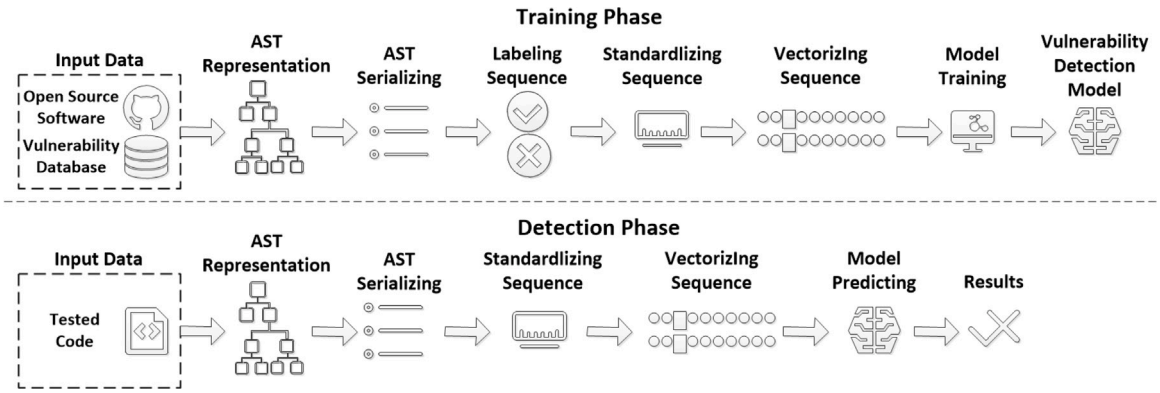


Fig. 6. The process of methods based on ASTs.

Pornprasit and Tantithamthavorn (2023) introduced DeepLineDP, a line-level vulnerability detection method, underscoring the hierarchical and context-dependent nature of source code. It notes that identical tokens can manifest divergent meanings across different lines of code. Current methods focusing solely on the relationships between distant token sequences overlook proximal relationships. DeepLineDP incorporates JavaParser for AST generation, integrates it with WordVec for vectorization, and employs a BGRU complemented by a hierarchical attention network to capture and learn the nuanced structure of code. Utilizing Wattanakriengkrai's Java vulnerability dataset for experiments, DeepLineDP demonstrates a significant uptick in accuracy and cost-effectiveness compared to prevailing methods.

4.5. Summary

Graph-based representation methods initially transform code into various graph-based forms, subsequently leveraged by DL models to elicit abstract features. These models are then enhanced with specific fully connected layers or integrated with traditional binary classification techniques to forge a framework adept at vulnerability detection. Graph representations, distinguished by their abstraction and complexity over token sequences, excel in maintaining the intricate structure of the code. The token sequences pre-processed lose the original formatting and complex architecture of the source code. The semantics gleaned from these sequences hinge on the token's sequential arrangement, thus typically capturing only linear semantics, which leads to a partial feature extraction based on token sequences. Conversely, graph representations adeptly illustrate many-to-many relationships among vertices (i.e., statements or tokens), mirroring the intricate structural interconnections within program elements.

ASTs present a hierarchical blueprint of source code, delineating the syntactic structure in a tree format. Within ASTs, each node symbolizes a syntactic unit of the code (such as expressions, statements, function definitions, etc.), with edges indicating relationships such as parent–child and sibling connections. ASTs serve as an abstract code representation beneficial for compilers, static analysis, and program comprehension. Current vulnerability detection methods that rely on AST representations predominantly utilize RNN models (mainly LSTM or GRU and their variations), rather than GNN models. This preference is attributed to ASTs' proclivity for reflecting the syntactic, rather than semantic, structure of source code, given that vulnerabilities often correlate more directly with semantic content, making GNN-based direct feature extraction from AST structures suboptimal. Consequently, the prevalent strategy employs AST-enhanced token sequences, traversing the AST of the source code to derive token sequences that embody more structural semantics, thereby streamlining the process and reducing costs.

The PDGs depict the execution dependencies within programs as graphical representations. In PDGs, nodes represent statements or expressions from the source code, while edges delineate dependency relationships among these entities, such as data, control, and output dependencies. PDGs excel in illustrating the multifaceted dependencies within programs, thereby facilitating more accurate code analysis and comprehension by static analysis tools.

The CPGs endeavors to encapsulate the maximum attainable program information and semantics within a unified structure. It is a layered, multidimensional graph that amalgamates ASTs, CFGs, and PDGs, further enriched with detailed property data. CPG nodes can denote functions, statements, variables, etc., each node with a unique identifier. The inter-node relationships encompass data edges, control edges, and call edges, representing variable dependencies, control dependencies, and function calls, respectively. Comparing the advantages and disadvantages of three graphs in graph-based vulnerability detection methods is shown in the Table 3.

In recent years, owing to the accelerated advancement of GPU technology, graph-based methods, which necessitate greater computational resources than token sequence-based methods, have garnered increased attention and development within the academic community. Modern GPUs, capable of processing thousands of nodes and edges in parallel, markedly enhance processing velocity and efficiency. This leap in hardware capabilities has catalyzed opportunities for both practical application and scholarly investigation of graph-based methodologies. Consequently, scholars are now equipped to conduct static vulnerability detection across expansive code bases with enhanced precision and speed.

Table 4 summarizes recent methods that use graph-based representations of source code, the DL models used, graph representations, Principles, vulnerability datasets, and graph transformation tools.

5. Other vulnerability detection methods

For the past few years, there has been a subset of studies employing techniques similar to those used in the aforementioned ML-based vulnerability detection methods, which hold significant referential value but fall outside the scope of the methodologies categorized in this paper. This section provides a unified introduction and analysis of these methods.

5.1. Non-machine learning source code vulnerability detection methods

To combat the pervasive spread of vulnerabilities attributable to code reuse across the internet, Li et al. (2016) presented VulPecker, a system designed to ascertain the presence of specific vulnerabilities within source code. Central to VulPecker's approach is the employment of a predefined feature set for characterizing code patches, alongside the strategic selection of code similarity algorithms tailored to distinct vulnerability traits. In its learning phase, VulPecker undertakes the identification of vulnerabilities by generating diff files from vulnerable and patched code, endeavoring to encapsulate vulnerability

Table 3
Comparison between 3 graphs.

Features	AST	PDG	CPG
Syntax Feature Capture Capability	Strong	Weak	Strong
Semantic Feature Representation Capability	Insufficient reflection of semantic information	Includes data flow and control flow dependencies	Include data flow, control flow dependencies, and call relationships
Structural Feature Capture Capability	Relatively strong (capable of capturing loops, conditional branches, variable declarations, etc.)	Relatively strong (capable of representing various dependencies)	Strong (includes structural, dependency, and execution path information)
Scalability	Weak	Relatively strong	Strong (nodes with rich attribute information)
Computational Cost	Weak	Relatively strong	Strong
Suitable DL Models	GNN Prefers RNN, not suitable for GNN	Prefers GNN	Prefers GNN

Table 4
Methods based on graphs.

Number	Literature	Year	DL method	Graph	Principles	Dataset	Graph tool
1	POSTER	2017	BLSTM	AST	AST To Sequence	POSTER	–
2	Cross-Project ...	2018	LSTM	AST	AST+depth-first traversal	POSTER	–
3	FUNDED	2020	GGNN	AST	AST+GNN+security-related code commits	–	–
4	Efficient vulnerability ...	2020	BGRU	AST	AST to Sequence+Pack-padded	SARD	–
5	DeepBalance	2020	BLSTM	AST	AST+Random Forests+fuzzy oversampling	POSTER	codeSensor
6	Automatic Feature Learning	2021	LSTM	AST	AST+Sequence+LSTM= FeaturesVectors(local Features) Cluster+codebook=global Features	–	–
7	Software Vulnerability ...	2021	BLSTM	AST	AST+Random Forest Classification	SARD+Real data	codeSensor
8	DeepLineDP	2023	BGRU	AST	AST+Hierarchical attention mechanisms	Wattanakrieng- krai Java Dataset	JavaParser
9	CPGVA	2018	LSTM	CPG	CPG+Word2Vec+LSTM	SARD	Joern
10	VulSniper	2019	Attention neural network	CPG	CPG+Attention mechanisms+SARD	SARD	Joern
11	VELVET	2022	GGN+Trans- former	CPG	Pre-training on artificial dataset+fine-tuning on real dataset	SARD+D2A	Joern
12	VulGDA	2023	GNNN	CPG	GGNN embedding+Transfer Learning	Devign+Reveal	Joern
13	IVDETECT	2021	GCN	PDG	GRU code representation with attention+GNN explainer explanatory models	Bigvul+Reveal +Devign	–
14	VulCNN	2022	CNN	PDG	Centrality analysis+Heat map	–	Joern
15	mVulPreter	2022	GGNN	PDG	Slice level(PDG,POI)+function level	Bigvul	–
16	Vulchecker	2022	GNN	ePDG	IR+Data Augment	SARD	Joern
17	LineVD	2022	GAT	PDG	codeBert participle	Bigvul	Joern
18		2017	GCN	CFG	CFG on IR	–	–
19	Devign	2019	Gated GRN	AST+CFG +DFG=CCG	Graph Wraparound Model+Graph Relational Embedding Attention	Devign	Joern
20	Hellendoorn et al.	2020	Transformer	PG	Graph Wraparound Model+Graph Relational Embedding Attention Transformer	–	–
21	BGNN4VD	2021	BGNN+CNN	CCG	BGNN learning features+CNN extracting features and classification	–	Joern
22	Deepwukong	2021	GCN/GAT/k-GNNs	POI+PDG =XFG	XFG+Doc2Vec	SARD	–
23	Vu1SPG	2021	r-GCN	SPG	SPG+Triple Attention Mechanism	SARD+NVD	Joern

characteristics within these diff files. Subsequently, it selects code similarity algorithms deemed efficacious for particular vulnerabilities. The algorithm's input encompasses candidate code similarity algorithms, vulnerability diff block feature vectors, an accuracy threshold, and the VCID database, with the outcome being a CVE algorithm mapping table. This methodology unfolds in three stages: initially, selecting code similarity algorithms capable of differentiating between vulnerable and patched code; next, identifying the code similarity algorithms most appropriate at the code snippet level; and finally, opting for the algorithm that exhibits the lowest false negative rate within the VCID library. During the experimental phase, VulPecker successfully identifies 40 vulnerabilities within Firefox, FFmpeg, and Qemu from a pool of 246 vulnerabilities not disclosed in the NVD vulnerability database.

Cui et al. (2021) introduced VulDetector, a vulnerability detection methodology predicated on the comparison of Weighted Feature Graphs (WFGs), aimed at mitigating the high false-negative rate and enhancing the accuracy. VulDetector commences by pinpointing and appraising the significance of key identifiers within diff files to select vulnerability-sensitive keywords. It proceeds to locate these keywords and corresponding statements within the source code, employing a breadth-first search within the CFGs to isolate sections bearing data and control dependencies on the vulnerability-sensitive keywords, thereby generating sub-CFGs. Then VulDetector enriches the CFGs by incorporating syntactic features derived from the AST into the CFG subgraphs. Finally, VulDetector culminates with the assignment of weights to each node within the CFG subgraphs, the creation of a WFG, and the application of bipartite graph matching to evaluate graph similarity, thus identifying potential vulnerabilities based on these similarity metrics. Experimental findings demonstrate that VulDetector surpasses the efficiency of existing methods.

Sun et al. (2021) introduced VDSimilar, a novel code vulnerability detection method based on metric learning, which aims to directly capture similarities from a vulnerability perspective. VDSimilar employs a classification model trained to compute similarity scores through metric learning, effectively differentiating between similar pairs (i.e., two vulnerable functions, labeled as '1') and dissimilar pairs (i.e., a vulnerable function alongside its patched counterpart, labeled as '0'). This method incorporates a Siamese network architecture augmented with an attention mechanism to discern similarity features from both similar and dissimilar pairs. During the test phase, various versions of vulnerable functions and their corresponding patched versions are collected to construct a dataset for vulnerability feature extraction. Experimental results demonstrate VDSimilar's superiority over several existing methods, evidenced by its higher detection accuracy and F1 scores.

Xiao et al. (2020) highlight that software systems frequently contain undiscovered duplicate vulnerabilities, often due to the reuse of code base or shared logic. In response, the authors propose a novel method, MVP, designed to diminish the false positive and false negative rates in vulnerability detection by amalgamating vulnerability and patch features. MVP identifies the syntax and semantics of vulnerable code statements through code slicing, utilizing the PDGs and ASTs for each function. It then transforms these code slices into signatures via a hashing algorithm and extracts similar signatures from patch functions. In its detection phase, MVP compares vulnerability feature signatures against patch feature signatures, focusing on functions that match vulnerability features but not patch features. The evaluation of MVP across ten open-source systems validates its efficacy, outperforming existing clone and function matching methods in identifying vulnerabilities that were overlooked by other techniques.

5.2. DL-based source code analysis methods

Zhang et al. (2019) identify two primary limitations in using deep neural networks for representing code with ASTs: the gradient vanishing issue, attributed to the substantial size of ASTs leading to a loss of long-term contextual code information, and the disruption of the original syntactic structure of source code when ASTs are transformed into or treated as binary trees, thereby impairing the network's capacity to comprehend complex code semantics. To overcome these challenges, the authors propose the ASTGNN method. ASTGNN segments the code's AST into smaller statement trees and leverages a recursive encoder to extract lexical and syntactic information, culminating in the generation of a code fragment vector through a BGRU. Code classification tests on the OJ dataset confirms that ASTGNN outshines competing methods.

Lin and Cai (2021) explore the performance of various neural network models in DL-based vulnerability detection. This study, utilizing the NVD dataset, involves processing source code into slices and vectorizing it to provide input data for the models. The evaluated models include LSTM, BLSTM, GRU, BGRU, CNN, and TCN. The results of the comparative experiments reveal that BLSTM achieves higher accuracy and precision compared to LSTM, while BGRU's improvements over GRU are minimal. When considering accuracy, F1 scores, the rates of false negatives and false positives, and precision, CNN demonstrates the most balanced performance.

Fu and Tantithamthavorn (2022) presented a Transformer-based method, LineVul, for line-level vulnerability prediction, aiming to enhance the existing IVDetect method. The study highlights limitations in the IVDetect method, including its dependence on specific dataset training, the RNN structure's challenge in capturing long-term dependencies, and the coarse granularity of subgraph explanations. LineVul utilizes CodeBERT with an attention mechanism to overcome these limitations. LineVul allows CodeBERT's attention layers to capture long-term dependencies effectively through dot-product operations. It moves away from specific dataset training to using vector representations of source code generated by CodeBERT. Moreover, CodeBERT's attention mechanism enables precise identification of vulnerable code lines. Testing on a large-scale dataset with 188k C/C++ functions indicates that LineVul significantly enhances F1 scores and top-10 accuracy at both function-level and line-level predictions.

Wattanakriengkrai et al. (2022) proposed the LINE-DP framework, integrating DL with model-agnostic techniques from the field of explainable artificial intelligence for line-level vulnerability detection. These model-agnostic techniques aim to enhance the interpretability of ML model outcomes by identifying important features in a given file and estimating the contribution of each token feature to the model predictions, irrespective of the model type. LINE-DP begins by constructing file-level models using code token features and then generating detection results for test files. Subsequently, it identifies key token features using Local Interpretable Model-Agnostic Explanations (LIME), a method based on diagnostic interpretability techniques. Vulnerable code lines are flagged according to risk rating. Evaluations conducted on nine Java open-source systems demonstrate that LINE-DP outstrips six comparative methods in prediction accuracy and performance.

Hu et al. (2023) introduced the AURC framework, which aims to detect vulnerabilities by cross-examining API documentation, API calls, and source code. AURC is comprised of six modules: Context-sensitive Backtrace Prediction (CBP), Cut-off Point Set (CoPS), Range Deduction Tree (RDT), a pre-trained classifier, mapping tables, and correctness inference. During the API calling process in code, three entities are involved: API documentation, the caller (code making the API call), and the callee (API's source code). The context-sensitive CBP analyzes the callee to deduce its return values. CoPS checks returns from the caller, identifying conditions within if statements and storing compared operands and symbols. RDT infers the range of return values based on conditions identified in CoPS. The pre-trained classifier is used to filter sentences in API documentation related to return values. Mapping tables then convert these sentences into vector forms for comparison and cross-examination among API documentation, code calling the API, and the API's source code, thereby identifying discrepancies. Finally, the correctness inference module uses four heuristic rules to identify flawed API calls. Evaluations on ten common code repositories reveal that AURC uncovers 529 new vulnerabilities and 224 new API documentation defects.

5.3. DL-based binary code analysis methods

Xu et al. (2017) introduced Gemini, a neural network-based graph embedding methodology designed for cross-platform binary code similarity detection. Gemini enhances the traditional CFG by incorporating intra-block and inter-block properties, resulting in an Attributed-CFG (ACFG). ACFGs serve as the input for an improved Structure2vec graph embedding network, which computes graph embedding vectors. To facilitate end-to-end training, Gemini employs a Siamese architecture comprising two identical neural networks, each processing an ACFG. An additional layer computes the cosine distance between the two embeddings. Experimental results demonstrate that Gemini achieves excellent performance in cross-platform binary code search.

Gao et al. (2018b) presented VulSee-ker, a semantic learning-based vulnerability detection method optimized for cross-platform binary code. VulSeeker enhances both accuracy and efficiency through the construction of labeled semantic flow graphs (LSFGs) and the utilization of deep neural networks for function semantics generation. The LSFG integrates CFG and DFG elements, with edges binary-labeled. Then VulSeeker extracts lightweight instruction features from each LSFG basic block, encodes them as numerical vectors, and inputs these vectors into a custom semantic-aware deep neural network model. The model iteratively computes binary function embedding vectors. The similarity between two binary functions is assessed using the cosine distance of their embedding vectors, determining the presence of known vulnerabilities. Experimental results indicate VulSeeker's superior performance compared to existing methodologies.

Gao et al. (2018a) introduced VulSeeker-Pro, an advanced binary vulnerability detection tool comprising two primary modules: the Semantic Learning Predictor and the Emulation Engine. The Semantic Learning Predictor utilizes the Gemini to generate semantic embedding vectors representing function semantics. Through iterative processing, Gemini produces embedding vectors that are aggregated into a 64-dimensional vector representing the entire function. Then the vector is used to compute inter-function similarity and predict candidate functions most similar to vulnerabilities in the target binary code. The Emulation Engine emulates the predicted candidate functions to obtain their semantic signatures, enabling more precise identification of vulnerability-similar functions. VulSeeker-Pro achieves enhanced detection accuracy through a two-stage similarity scoring process, maintaining time efficiency comparable to standalone semantic learning methods. Experimental results demonstrate VulSeeker-Pro's significant performance improvements over the existing Gemini in vulnerability detection accuracy.

To address the limitations of existing methods in cross-platform binary code semantic equivalence measurement, (Yang et al., 2021) proposed Asteria, a deep learning-based AST encoding method. Asteria leverages Tree-LSTM network, a variant of recursive neural networks, to learn function semantic representations from ASTs. The Tree-LSTM network processes tree-structured inputs by computing each non-leaf node's information using its children's data. Binary code similarity detection is subsequently performed by measuring the similarity between representation vectors. Asteria is trained on a substantial dataset of 1,022,616 function pairs and evaluated on 95,078 function pairs, demonstrating significant advantages over Gemini in binary similarity detection. In the practical application, Asteria successfully identified 75 vulnerable functions across 5,979 firmware images, highlighting its efficacy in vulnerability search tasks.

Pei et al. (2020) proposed TREX, a deep learning-based binary code similarity detection method that learns execution semantics from micro-traces for binary similarity matching and vulnerability function detection. TREX initially collects micro-execution traces of binary code through dynamic analysis and symbolic execution, capturing instruction execution order, and read/write operations on registers and memory. The collected micro-traces are then analyzed using a multi-head self-attention mechanism to capture long-range dependencies within instruction sequences. To handle cross-architecture scenarios, TREX employs a Siamese network architecture to enhance the robustness of learned semantics. The similarity between learned execution semantic representations is computed to assess instruction similarity. Experiments on cross-architecture and compiler datasets demonstrate TREX's effectiveness. And TREX discovered 16 previously undisclosed vulnerabilities in 180 recent firmware images, validating its feasibility in vulnerability detection.

To address Asteria's limitations in terms of high computational costs and insufficient accuracy in large-scale firmware bug searches, Yang et al. (2024) proposed Asteria-Pro, an enhanced deep learning architecture incorporating domain knowledge-based pre-filtering and re-ranking modules. Asteria-Pro's key advancements include a pre-filtering module to eliminate dissimilar functions, thereby reducing subsequent deep learning model computations, and a re-ranking module to refine the ranking of candidate functions generated by the deep learning model. Experimental results demonstrate that Asteria-Pro's pre-filtering module reduces computation time by 96.9%, while the re-ranking module improves the Mean Reciprocal Rank by 23.71% and recall by 36.4%. The integration of these modules enables Asteria-Pro to significantly outperform state-of-the-art methods in bug search tasks, successfully detecting 1,482 vulnerable functions with 91.65% accuracy in real-world firmware vulnerability searches.

Abu-Mahfouz et al. (2024) presented a novel CNN-based methodology for detecting vulnerabilities in router firmware by transforming firmware into image representations and employing image analysis techniques for classification. The process begins with converting firmware byte strings into gray-scale and RGB image sets using a custom image converter script. Automatic image filtering is performed using Histogram of Oriented Gradients (HOG), Local Binary Patterns (LBP), and Gabor filters. Then six neural network models are developed and evaluated on different image filter sets to identify the optimal filter and model combination. The trained CNN models then identify salient features in the router firmware images, classifying them as vulnerable or secure. Experimental evaluation on a manually curated dataset comprising over 2,500 test samples demonstrates that the HOG filter achieved the highest accuracy, with an average accuracy of 85.81% across all tests and models, and a peak accuracy of 97.94% when used in conjunction with the best-performing CNN model.

To address the issues of low precision and poor scalability in existing pseudocode difference analysis and binary code analysis methods, Gao et al. (2024) proposed SigmaDiff, a semantically aware graph matching model. SigmaDiff first constructs inter-procedural dependency graphs (iPDGs) at the code IR level, then employs lightweight symbolic analysis to extract initial node features. Subsequently, SigmaDiff utilizes the Deep Graph Matching Consensus (DGMC) model for graph structure matching, incorporating a novel loss function that combines data and opcode type constraints, while adopting pre-training and fine-tuning strategies to accelerate the graph matching process. Ultimately, SigmaDiff matches corresponding code tokens by aligning IR-related variables, achieving more accurate and efficient binary code comparison and pseudocode difference analysis. In experimental evaluations, SigmaDiff outperforms existing difference comparison tools in cross-version, cross-optimization level, cross-compiler, and cross-architecture difference comparisons. Furthermore, SigmaDiff demonstrates excellent performance in real-world CVE patch detection.

5.4. Summary

The technical route and method principles of the methods mentioned in this section are shown in Table 5.

Table 5
Other vulnerability detection methods.

Number	Literature	Year	Principles	Technical route
1	VulPecker	2016	Based on similarity	Based on code patch diff file
2	Gemini	2017	Binary code similarity analysis	ACFG + Structure2vec + Siamese network
3	VulSeeker	2018	Binary code similarity analysis	LSFG + deep neural networks
4	VulSeeker-Pro	2018	Binary code similarity analysis	Improvements to VulSeeker + Gemini + emulation engine
5	A Novel Neural ...	2019	ML-based code cloning and categorization	Code classification and clone detection + AST + GRU
6	Vuldetector	2020	Based on similarity	Based on code patch diff file + slicing on CFG based on POIs, calculate weights for nodes to get WFG
7	An empirical study ...	2021	DL model performance comparison	Investigate how effective different neural network models are on DL-based vulnerability detection
8	Asteria	2021	Binary code similarity analysis	AST + Tree-LSTM
9	TREX	2021	Binary code similarity analysis	multi-head self-attention mechanism + Siamese network
10	VDSimilar	2021	Based on similarity	Siamese Network Architecture + Attention Mechanisms + Simultaneous Learning from Similar and Dissimilar Pairs
11	LineVul	2022	Specialized improvements	Improvements to IVDETECT + codeBert vectorization, adding positional coding when functions are coded, predictions with CodeBERT
12	Predicting Defective Lines ...	2022	Model-based agnostic techniques	Interpretation of model prediction results based on model agnostic techniques + Local Interpretable Model-Agnostic Explanations
13	AURC	2022	Based on API	Check defective API calls; Bert text analysis
14	MVP	2023	Based on similarity	Similarity-based checking for duplicate vulnerabilities + vulnerability patch based; PDG + AST + hash = signature
15	Asteria-Pro	2024	Binary code similarity analysis	Improvements to Asteria + pre-filtering + re-ranking
16	A Deep Learning Approach ...	2024	Based on image analysis	CNN + image filtering
17	SigmaDiff	2024	Binary code similarity analysis	IR-based + iPDGs + DGMC

6. Conclusion

6.1. Limitations of current methods

As ML techniques and code preprocessing technologies continue to evolve, ML-based static vulnerability detection methods have been progressively advancing. Despite the steady improvement in experimental outcomes across various methods, numerous challenges still remain to be addressed in this field:

6.1.1. Dependency on large quantities of high-quality labeled data

ML methods, particularly DL, rely heavily on large volumes of accurately labeled and representative data. In the realm of cyberspace security, despite the existence of public vulnerability databases such as Common Vulnerabilities and Exposures (CVE) and National Vulnerability Database (NVD), these databases primarily serve vulnerability identification and documentation, often suffering from inconsistent annotation quality, labeling errors, and incomplete coverage. Annotating these data for comprehensive vulnerability datasets suitable for ML training demands substantial effort. Moreover, due to privacy and confidentiality concerns, some software vendors may withhold details of real vulnerabilities, compromising the completeness of vulnerability datasets. Consequently, besides real-world vulnerability datasets, some methods attempt to train ML models using publicly available, artificially constructed datasets. Furthermore, issues such as noise in datasets and data imbalance can also impact the final detection performance of ML models. Noise in datasets may include spelling errors in code, annotation errors, or complex programming conventions, potentially impairing the model's learning ability and reducing accuracy. Static vulnerability detection datasets are often imbalanced, with fewer instances of vulnerabilities (positive class) compared to non-vulnerable code (negative class). Such severe class imbalance can lead to models being biased towards the majority class, thereby overlooking the less frequent but more critical positive class samples. Scholars typically mitigate this issue through resampling, cost-sensitive learning, or adaptive loss functions, yet the fundamental challenge posed by data imbalance remains unresolved.

6.1.2. Poor interpretability of detection results

Despite the powerful capabilities of ML, especially DL, in handling complex features and large-scale data, the decision-making processes of these models are seldom transparent, particularly for DL neural networks with complex internal structures and multiple layers. The opacity of models leads to low confidence among cybersecurity professionals in the detection results, making it difficult to understand why certain instances are classified as vulnerable. In the field of cyberspace security, interpretability is crucial, as security professionals need to take appropriate measures based on the causes of vulnerabilities. The lack

of interpretability necessitates complex security audits and analyses of code by cybersecurity professionals, further consuming significant human and material resources.

6.1.3. Generalization issues across different languages and types of vulnerabilities

Generalization is one of the core issues concerning the utility of ML models. Vulnerability detection models that perform well on datasets of specific languages or types of vulnerabilities may struggle with programming languages of other kinds or new categories of vulnerabilities. This challenge demands that vulnerability detection models not only be flexible enough to adapt to different code representations and language structures but also capable of effective detection in complex, evolving, and unknown coding environments. Transfer learning and multitask learning are potential strategies to address this issue, yet designing vulnerability detection models adaptable to various changes remains an open research area.

6.1.4. Performance and cost trade-offs in source code preprocessing

ML-based methods for detecting vulnerabilities necessitate traditional source code preprocessing steps, including the conversion of source code into token sequences and assorted graph-based representations. To ensure richness of information and model accuracy, the preprocessing process inevitably needs to mine various semantic and syntactic features of code, a process that is typically computation-intensive. The complexity of these features also consumes more time and resources during model training. Clearly, there is a balance to be struck between the complexity of preprocessing and the computational resources required for model training. Excessive preprocessing may lead to high computational costs, while insufficient preprocessing can result in performance degradation. Identifying the optimal source code representation and feature extraction method requires a balance between theoretical research and practical considerations.

6.2. Future perspectives

In the area of ML-based source code security analysis, particularly ML-based static vulnerability detection techniques, there are several promising directions that could advance the field:

6.2.1. Construction and improvement of vulnerability datasets

Future research should focus on the ongoing development and refinement of large-scale, high-quality, finely detailed, and precisely annotated datasets that should encompass a broad spectrum of vulnerability types and programming languages. The current industry method to building vulnerability datasets focuses on collecting and manually annotating real-world security vulnerabilities and their fixes, a process that requires significant manpower and resources. The exploration of automated mechanisms for the collection and annotation of vulnerability data is an essential direction for future work. Moreover, there are endeavors aimed at enlarging vulnerability datasets through data augmentation techniques, such as integrating typical real-world vulnerable code snippets with open-source projects. This method can balance the distribution of different types of vulnerabilities. And experimental results suggest a slight advantage over methods based entirely on artificially synthesized datasets. Therefore, exploring more precise and ingenious data augmentation techniques is also a viable direction for the construction and improvement of vulnerability datasets.

6.2.2. More effective code representation and preprocessing methods

Whether based on token sequences or graph-based approaches, deriving succinct code representations that accurately capture semantic features, especially vulnerability characteristics, is a key future research direction. Advanced code representation methods, such as CPG, PDG, and IR-based CPG representations, have emerged, but no single representation that combines the advantages of all types exists yet. Methods combining multiple representations, such as Devign and BGNN4VD, could theoretically capture a broader range of code vulnerability characteristics, leading to more comprehensive performance. However, complex and precise representations usually come at the cost of increased preprocessing steps and computational complexity. Therefore, balancing the breadth and depth of vulnerability detection with computational resources according to specific needs is another important future direction.

6.2.3. Optimization of ML models

Beyond source code preprocessing, designing and optimizing existing ML models, particularly DL neural networks, is a key step in capturing and understanding code language features. DL models can improve vulnerability detection performance by adding deeper network structures. Thus, arranging additional hidden layers and convolutional layers to capture vulnerability code characteristics is a viable direction. In the natural language processing field, rapidly evolving pre-trained models such as BERT and GPT have become hot research topics. Transferring such pre-trained models to static vulnerability detection tasks to obtain better code representations and features is also a key future research direction.

6.2.4. Exploration of cross-language vulnerability detection methods

Researching a unified representation for multiple programming languages and developing a vulnerability detection framework and methods applicable across languages is a current hot topic in the field. The academic community is dedicated to abstracting source code into a common intermediate representation and further preprocessing this representation for model training. Besides a common intermediate representation, combining federated learning methods and utilizing data and different feature learning techniques from various programming languages also has significant research value.

6.2.5. Vulnerability detection and automated repair suggestions

The purpose of vulnerability detection is to maintain software system security. Therefore, providing actionable repair suggestions based on the code context and vulnerability features, alongside detection, is a key future research direction. This approach might involve integrating finer-grained code context analysis with code generation technologies. Future efforts could combine static code analysis with dynamic behavior analysis to form a more comprehensive security assessment. Meanwhile, the ML models could be used to score and rank possible repair solutions for offering developers the most appropriate repair suggestions.

CRediT authorship contribution statement

Chen Liang: Validation, Investigation, Formal analysis, Data curation, Conceptualization. **Qiang Wei:** Methodology, Funding acquisition. **Jiang Du:** Investigation, Conceptualization. **Yisen Wang:** Funding acquisition, Formal analysis, Conceptualization. **Zirui Jiang:** Formal analysis, Data curation.

Declaration of competing interest

We declare that we have no financial and personal relationships with other people or organizations that can inappropriately influence our work, there is no professional or other personal interest of any nature or kind in any product, service and/or company that could be construed as influencing the position presented in, or the review of, the manuscript entitled.

Data availability

No data was used for the research described in the article.

References

Abu-Mahfouz, A., Alrabaee, S., Khasawneh, M., Gergely, M., Choo, K.K.R., 2024. A deep learning approach to discover router firmware vulnerabilities. IEEE Trans. Ind. Inform. 20 (1), 691–702. http://dx.doi.org/10.1109/TII.2023.3269774.

Acharya, M., Xie, T., Pei, J., Xu, J., 100. Mining API patterns as partial orders from source code: from usage scenarios to specifications. In: Proceedings of the the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering.

Alhuzali, A., Gjomemo, R., Eshete, B., 2018. NAVEX: Precise and scalable exploit generation for dynamic web applications. In: 27th USENIX Security Symposium.

Black, P.E., 2018. A software assurance reference dataset: Thousands of programs with known bugs. J. Res. Natl. Inst. Stand. Technol. 1–3.

Cao, S., Sun, X., Bo, L., Wei, Y., Li, B., 2021. BGNN4vd: Constructing bidirectional graph neural-network for vulnerability detection. Inf. Softw. Technol. 136, 106576. http://dx.doi.org/10.1016/j.infsof.2021.106576, URL https://linkinghub.elsevier.com/retrieve/pii/S0950584921000586.

Chakraborty, S., Krishna, R., Ding, Y., Ray, B., 2022. Deep learning based vulnerability detection: Are we there yet? IEEE Trans. Softw. Eng. 48 (9), 3280–3296. http://dx.doi.org/10.1109/TSE.2021.3087402, URL https://ieeexplore.ieee.org/document/9448435/.

Cheng, X., Wang, H., Hua, J., Xu, G., Sui, Y., 2021. DeepWukong: Statically detecting software vulnerabilities using deep graph neural network. ACM Trans. Softw. Eng. Methodol. 30 (3), 1–33. http://dx.doi.org/10.1145/3436877, URL https://dl.acm.org/doi/10.1145/3436877.

Chowdhury, I., Zulkernine, M., 2011. Using complexity, coupling, and cohesion metrics as early indicators of vulnerabilities. J. Syst. Archit. (No.3), 294–313.

Croft, R., Xie, Y., Babar, M.A., 2023. Data preparation for software vulnerability prediction: A systematic literature review. IEEE Trans. Softw. Eng. 49 (3), 1044–1063. http://dx.doi.org/10.1109/TSE.2022.3171202, URL https://ieeexplore.ieee.org/document/9765337/.

Cui, L., Hao, Z., Jiao, Y., Fei, H., Yun, X., 2021. VulDetector: Detecting vulnerabilities using weighted feature graph comparison. IEEE Trans. Inf. Forensics Secur. 16, 2004–2017. http://dx.doi.org/10.1109/TIFS.2020.3047756, URL https://ieeexplore.ieee.org/document/9309254/.

Dam, H.K., Tran, T., Pham, T., Ng, S.W., Grundy, J., Ghose, A., 2021. Automatic feature learning for predicting vulnerable software components. IEEE Trans. Softw. Eng. 47 (1).

Ding, Y., Suneja, S., Zheng, Y., Laredo, J., Morari, A., Kaiser, G., Ray, B., 2022. VELVET: a noVel ensemble learning approach to automatically locate VulnErable sTatements. In: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering. SANER, IEEE, Honolulu, HI, USA, pp. 959–970. http://dx.doi.org/10.1109/SANER53432.2022.00114, URL https://ieeexplore.ieee.org/document/9825786/.

Doyle, M., Walden, J., 2011. An empirical study of the evolution of PHP web application security. In: 2011 Third International Workshop on Security Measurements and Metrics. IEEE, Baniff, Alberta, Canada, pp. 11–20. http://dx.doi.org/10.1109/Metrisec.2011.18, URL http://ieeexplore.ieee.org/document/6165758/.

Duan, X., Wu, J., Ji, S., Rui, Z., Luo, T., Yang, M., Wu, Y., 2019. VulSniper: Focus your attention to shoot fine-grained vulnerabilities. In: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence. International Joint Conferences on Artificial Intelligence Organization, Macao, China, pp. 4665–4671. http://dx.doi.org/10.24963/ijcai.2019/648, URL https://www.ijcai.org/proceedings/2019/648.

Fan, J., Li, Y., Wang, S., Nguyen, T.N., 2020. A C/C++ code vulnerability dataset with code changes and CVE summaries. In: Proceedings of the 17th International Conference on Mining Software Repositories. ACM, Seoul Republic of Korea, pp. 508–512. http://dx.doi.org/10.1145/3379597.3387501, URL https://dl.acm.org/doi/10.1145/3379597.3387501.

Feng, H., Fu, X., Sun, H., Wang, H., Zhang, Y., 2020a. Efficient vulnerability detection based on abstract syntax tree and deep learning. In: IEEE INFOCOM 2020 - IEEE Conference on Computer Communications Workshops. INFOCOM WKSHPS, IEEE, Toronto, ON, Canada, pp. 722–727. http://dx.doi.org/10.1109/INFOCOMWKSHPS50562.2020.9163061, URL https://ieeexplore.ieee.org/document/9163061/.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M., 2020b. CodeBERT: A pre-trained model for programming and natural languages. In: Findings of the Association for Computational Linguistics. EMNLP 2020, Association for Computational Linguistics, Online, pp. 1536–1547. http://dx.doi.org/10.18653/v1/2020.findings-emnlp.139, URL https://www.aclweb.org/anthology/2020.findings-emnlp.139.

Fu, M., Tantithamthavorn, C., 2022. LineVul: a transformer-based line-level vulnerability prediction. In: Proceedings of the 19th International Conference on Mining Software Repositories. ACM, Pittsburgh Pennsylvania, pp. 608–620. http://dx.doi.org/10.1145/3524842.3528452, URL https://dl.acm.org/doi/10.1145/3524842.3528452.

Gan, S., Zhang, C., Qin, X., Tu, X., Li, K., Pei, Z., Chen, Z., 2022. Path sensitive fuzzing for native applications. IEEE Trans. Dependable Secure Comput. (No.3), 1544–1561.

Gao, L., Qu, Y., Yu, S., Duan, Y., Yin, H., 2024. SigmaDiff: Semantics-aware deep graph matching for pseudocode diffing. In: Proceedings 2024 Network and Distributed System Security Symposium. Internet Society, San Diego, CA, USA, http://dx.doi.org/10.14722/ndss.2024.23208, URL https://www.ndss-symposium.org/wp-content/uploads/2024-208-paper.pdf.

Gao, J., Yang, X., Fu, Y., Jiang, Y., Shi, H., Sun, J., 2018a. VulSeeker-pro: enhanced semantic learning based binary vulnerability seeker with emulation. In: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, Lake Buena Vista FL USA, pp. 803–808. http://dx.doi.org/10.1145/3236024.3275524, URL https://dl.acm.org/doi/10.1145/3236024.3275524.

Gao, J., Yang, X., Fu, Y., Jiang, Y., Sun, J., 2018b. VulSeeker: a semantic learning based vulnerability seeker for cross-platform binary. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering. ACM, Montpellier France, pp. 896–899. http://dx.doi.org/10.1145/3238147.3240480, URL https://dl.acm.org/doi/10.1145/3238147.3240480.

Garg, A., Degiovanni, R., Jimenez, M., Cordy, M., Papadakis, M., Le Traon, Y., 2022. Learning from what we know: How to perform vulnerability prediction using noisy historical data. Empir. Softw. Eng. 27 (7), 169. http://dx.doi.org/10.1007/s10664-022-10197-4, URL https://link.springer.com/10.1007/s10664-022-10197-4.

Gkortzis, A., Mitropoulos, D., Spinellis, D., 2018. VulinOSS: a dataset of security vulnerabilities in open-source systems. In: Proceedings of the 15th International Conference on Mining Software Repositories. ACM, Gothenburg Sweden, pp. 18–21. http://dx.doi.org/10.1145/3196398.3196454, URL https://dl.acm.org/doi/10.1145/3196398.3196454.

Hellendoorn, V.J., Maniatis, P., Singh, R., Sutton, C., Bieber, D., 2020. Global relational models of source code.

Hin, D., Kan, A., Chen, H., Babar, M.A., 2022. LineVD: statement-level vulnerability detection using graph neural networks. In: Proceedings of the 19th International Conference on Mining Software Repositories. ACM, Pittsburgh Pennsylvania, pp. 596–607. http://dx.doi.org/10.1145/3524842.3527949, URL https://dl.acm.org/doi/10.1145/3524842.3527949.

Hu, P., Liang, R., Cao, Y., Chen, K., Zhang, R., 2023. AURC: detecting errors in program code and documentation. In: Calandrino, J.A., Troncoso, C. (Eds.), 32nd USENIX Security Symposium. USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023, USENIX Association, pp. 1415–1432, URL https://www.usenix.org/conference/usenixsecurity23/presentation/hu.

Jang, J., Agrawal, A., Brumley, D., 2012. ReDeBug: Finding unpatched code clones in entire OS distributions. In: 2012 IEEE Symposium on Security and Privacy. IEEE, San Francisco, CA, USA, pp. 48–62. http://dx.doi.org/10.1109/SP.2012.13, URL http://ieeexplore.ieee.org/document/6234404/.

Karim, R., Tip, F., Sochurkova, A., Sen, K., 2018. Platform-independent dynamic taint analysis for JavaScript. IEEE Trans. Softw. Eng. (No.12), 1.

Kim, S., Woo, S., Lee, H., Oh, H., 2017. VUDDY: A scalable approach for vulnerable code clone discovery. In: 2017 IEEE Symposium on Security and Privacy. SP, IEEE, San Jose, CA, USA, pp. 595–614. http://dx.doi.org/10.1109/SP.2017.62, URL http://ieeexplore.ieee.org/document/7958600/.

Krsul, I.V., 1998. Software Vulnerability Analysis. Purdue University.

Lee, M., Cho, S., Jang, C., Park, H., Choi, E., 2006. A rule-based security auditing tool for software vulnerability detection. In: 2006 International Conference on Hybrid Information Technology. IEEE, Cheju Island, pp. 505–512. http://dx.doi.org/10.1109/ICHIT.2006.253653, URL http://ieeexplore.ieee.org/document/4021258/.

Li, R., Feng, C., Zhang, X., Tang, C., 2019. A lightweight assisted vulnerability discovery method using deep neural networks. IEEE Access 7, 80079–80092. http://dx.doi.org/10.1109/ACCESS.2019.2923227, URL https://ieeexplore.ieee.org/document/8736948/.

Li, L., Feng, H., Zhuang, W., Meng, N., Ryder, B., 2017. CCLearner: A deep learning-based clone detection approach. In: 2017 IEEE International Conference on Software Maintenance and Evolution. ICSME, IEEE, Shanghai, pp. 249–260. http://dx.doi.org/10.1109/ICSME.2017.46, URL http://ieeexplore.ieee.org/document/8094426/.

Li, H., Kim, T., Bat-Erdene, M., Lee, H., 2013. Software vulnerability detection using backward trace analysis and symbolic execution. In: 2013 International Conference on Availability, Reliability and Security.

Li, Y., Wang, S., Nguyen, T.N., 2021. Vulnerability detection with fine-grained interpretations. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, Athens Greece, pp. 292–303. http://dx.doi.org/10.1145/3468264.3468597, URL https://dl.acm.org/doi/10.1145/3468264.3468597.

Li, X., Xin, Y., Zhu, H., Yang, Y., Chen, Y., 2023. Cross-domain vulnerability detection using graph embedding and domain adaptation. Comput. Secur. 125, 103017. http://dx.doi.org/10.1016/j.cose.2022.103017, URL https://linkinghub.elsevier.com/retrieve/pii/S0167404822004096.

Li, Z., Zou, D., Xu, S., Chen, Z., Zhu, Y., Jin, H., 2022a. VulDeeLocator: A deep learning-based fine-grained vulnerability detector. IEEE Trans. Dependable Secure Comput. 19 (4), 2821–2837. http://dx.doi.org/10.1109/TDSC.2021.3076142, URL https://ieeexplore.ieee.org/document/9416836/.

Li, Z., Zou, D., Xu, S., Jin, H., Qi, H., Hu, J., 2016. VulPecker: an automated vulnerability detection system based on code similarity analysis. In: Proceedings of the 32nd Annual Conference on Computer Security Applications. ACM, Los Angeles California USA, pp. 201–213. http://dx.doi.org/10.1145/2991079.2991102, URL https://dl.acm.org/doi/10.1145/2991079.2991102.

Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z., 2022b. SySeVR: A framework for using deep learning to detect software vulnerabilities. IEEE Trans. Dependable Secure Comput. 19 (4), 2244–2258. http://dx.doi.org/10.1109/TDSC.2021.3051525, URL https://ieeexplore.ieee.org/document/9321538/.

Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y., 2018. VulDeePecker: A deep learning-based system for vulnerability detection. In: Proceedings 2018 Network and Distributed System Security Symposium. Internet Society, San Diego, CA, http://dx.doi.org/10.14722/ndss.2018.23158, URL https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_03A-2_Li_paper.pdf.

Lin, W., Cai, S., 2021. An empirical study on vulnerability detection for source code software based on deep learning. In: 2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion. QRS-C, IEEE, Hainan, China, pp. 1159–1160. http://dx.doi.org/10.1109/QRS-C55045.2021.00173, URL https://ieeexplore.ieee.org/document/9741942/.

Lin, G., Zhang, J., Luo, W., Pan, L., De Vel, O., Montague, P., Xiang, Y., 2021. Software vulnerability discovery via learning multi-domain knowledge bases. IEEE Trans. Dependable Secure Comput. 18 (5), 2469–2485. http://dx.doi.org/10.1109/TDSC.2019.2954088, URL https://ieeexplore.ieee.org/document/8906156/.

Lin, G., Zhang, J., Luo, W., Pan, L., Xiang, Y., 2017. POSTER: Vulnerability discovery with function representation learning from unlabeled projects. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. ACM, Dallas Texas USA, pp. 2539–2541. http://dx.doi.org/10.1145/3133956.3138840, URL https://dl.acm.org/doi/10.1145/3133956.3138840.

Lin, G., Zhang, J., Luo, W., Pan, L., Xiang, Y., De Vel, O., Montague, P., 2018. Cross-project transfer representation learning for vulnerable function discovery. IEEE Trans. Ind. Inform. 14 (7), 3289–3297. http://dx.doi.org/10.1109/TII.2018.2821768, URL https://ieeexplore.ieee.org/document/8329207/.

Liu, S., Lin, G., Han, Q.L., Wen, S., Zhang, J., Xiang, Y., 2019. DeepBalance: Deep-learning and fuzzy oversampling for vulnerability detection. IEEE Trans. Fuzzy Syst. 1. http://dx.doi.org/10.1109/TFUZZ.2019.2958558, URL https://ieeexplore.ieee.org/document/8930093/.

Mccabe, T.J., 1976. A complexity measure.

Mirsky, Y., Macon, G., Brown, M., Yagemann, C., Pruett, M., Downing, E., Mertoguno, S., Lee, W., 2023. VulChecker: Graph-based vulnerability localization in source code. In: 31st USENIX Security Symposium, Security 2022.

Nagappan, N., Ball, T., 2005. Use of relative code churn measures to predict system defect density. In: Proceedings. 27th International Conference on Software Engineering, 2005. ICSE 2005.. IEEe, St. Louis, MO, USA, pp. 284–292. http://dx.doi.org/10.1109/ICSE.2005.1553571, URL http://ieeexplore.ieee.org/document/1553571/.

Neuhaus, S., Zimmermann, T., Holler, C., Zeller, A., 2007. Predicting vulnerable software components. In: Ning, P., Vimercati, S.D.C.d., Syverson, P.F. (Eds.), Proceedings of the 2007 ACM Conference on Computer and Communications Security, CCS 2007, Alexandria, Virginia, USA, October 28-31, 2007. ACM, pp. 529–540. http://dx.doi.org/10.1145/1315245.1315311.

Ozment, A., 2007. Improving vulnerability discovery models. In: Proceedings of the 3th ACM Workshop on Quality of Protection. QoP 2007, Alexandria, VA, USA, October 29, 2007.

Pang, Y., Xue, X., Namin, A.S., 2015. Predicting vulnerable software components through N-gram analysis and statistical feature selection. In: 2015 IEEE 14th International Conference on Machine Learning and Applications. ICMLA, IEEE, Miami, FL, USA, pp. 543–548. http://dx.doi.org/10.1109/ICMLA.2015.99, URL http://ieeexplore.ieee.org/document/7424372/.

Pei, K., Yang, J., Xuan, Z., Jana, S., 2020. TREX: Learning execution semantics from micro-traces for binary similarity.

Pham, V.T., Boehme, M., Santosa, A.E., Caciulescu, A.R., Roychoudhury, A., 2020. Smart greybox fuzzing. IEEE Trans. Softw. Eng. 1. http://dx.doi.org/10.1109/TSE.2019.2941681, URL https://ieeexplore.ieee.org/document/8839290/.

Plate, H., Ponta, S.E., Sabetta, A., 2015. Impact assessment for vulnerabilities in open-source software libraries. In: 2015 IEEE International Conference on Software Maintenance and Evolution. ICSME, IEEE, Bremen, Germany, pp. 411–420. http://dx.doi.org/10.1109/ICSM.2015.7332492, URL http://ieeexplore.ieee.org/document/7332492/.

Pornprasit, C., Tantithamthavorn, C.K., 2023. DeepLineDP: Towards a deep learning approach for line-level defect prediction. IEEE Trans. Softw. Eng. 49 (1), 84–98. http://dx.doi.org/10.1109/TSE.2022.3144348, URL https://ieeexplore.ieee.org/document/9689967/.

Radatz, J., Geraci, A., Katki, F., 1990. IEEE Standard Glossary of Software Engineering Terminology. The Institute of Electrical and Electronics Engineers.

Russell, R.L., Kim, L., Hamilton, L.H., Lazovich, T., Harer, J.A., Ozdemir, O., Ellingwood, P.M., McConley, M.W., 2018. Automated Vulnerability Detection in Source Code Using Deep Representation Learning.

Sajnani, H., Saini, V., Svajlenko, J., Roy, C.K., Lopes, C.V., 2016. SourcerCC: scaling code clone detection to big-code. In: Proceedings of the 38th International Conference on Software Engineering. ACM, Austin Texas, pp. 1157–1168. http://dx.doi.org/10.1145/2884781.2884877, URL https://dl.acm.org/doi/10.1145/2884781.2884877.

Scandariato, R., Walden, J., Hovsepyan, A., Joosen, W., 2014. Predicting vulnerable software components via text mining. IEEE Trans. Softw. Eng. 40 (10), 993–1006. http://dx.doi.org/10.1109/TSE.2014.2340398, URL http://ieeexplore.ieee.org/document/6860243/.

Sun, H., Cui, L., Li, L., Ding, Z., Hao, Z., Cui, J., Liu, P., 2021. VDSimilar: Vulnerability detection based on code similarity of vulnerabilities and patches. Comput. Secur. 110, 102417. http://dx.doi.org/10.1016/j.cose.2021.102417, URL https://linkinghub.elsevier.com/retrieve/pii/S0167404821002418.

Viet Phan, A., Le Nguyen, M., Thu Bui, L., 2017. Convolutional neural networks over control flow graphs for software defect prediction. In: 2017 IEEE 29th International Conference on Tools with Artificial Intelligence (ICTAI). IEEE, Boston, MA, pp. 45–52. http://dx.doi.org/10.1109/ICTAI.2017.00019, URL https://ieeexplore.ieee.org/document/8371922/.

Wang, S., Liu, T., Tan, L., 2016. Automatically learning semantic features for defect prediction. In: Proceedings of the 38th International Conference on Software Engineering. ACM, Austin Texas, pp. 297–308. http://dx.doi.org/10.1145/2884781.2884804, URL https://dl.acm.org/doi/10.1145/2884781.2884804.

Wang, P., Svajlenko, J., Wu, Y., Xu, Y., Roy, C.K., 2018. CCAligner: a token based large-gap clone detector. In: Proceedings of the 40th International Conference on Software Engineering. ACM, Gothenburg Sweden, pp. 1066–1077. http://dx.doi.org/10.1145/3180155.3180179, URL https://dl.acm.org/doi/10.1145/3180155.3180179.

Wang, H., Ye, G., Tang, Z., Tan, S.H., Huang, S., Fang, D., Feng, Y., Bian, L., Wang, Z., 2021. Combining graph-based learning with automated data collection for code vulnerability detection. IEEE Trans. Inf. Forensics Secur. 16, 1943–1958. http://dx.doi.org/10.1109/TIFS.2020.3044773, URL https://ieeexplore.ieee.org/document/9293321/.

Wartschinski, L., Noller, Y., Vogel, T., Kehrer, T., Grunske, L., 2022. VUDENC: Vulnerability detection with deep learning on a natural codebase for python. Inf. Softw. Technol. 144, 106809. http://dx.doi.org/10.1016/j.infsof.2021.106809, URL https://linkinghub.elsevier.com/retrieve/pii/S0950584921002421.

Wattanakriengkrai, S., Thongtanunam, P., Tantithamthavorn, C., Hata, H., Matsumoto, K., 2022. Predicting defective lines using a model-agnostic technique. IEEE Trans. Softw. Eng. 48 (5), 1480–1496. http://dx.doi.org/10.1109/TSE.2020.3023177, URL https://ieeexplore.ieee.org/document/9193975/.

Wu, Y., Zou, D., Dou, S., Yang, W., Xu, D., Jin, H., 2022. VulCNN: an image-inspired scalable vulnerability detection system. In: Proceedings of the 44th International Conference on Software Engineering. ACM, Pittsburgh Pennsylvania, pp. 2365–2376. http://dx.doi.org/10.1145/3510003.3510229, URL https://dl.acm.org/doi/10.1145/3510003.3510229.

Xiao, Y., Chen, B., Yu, C., Xu, Z., Yuan, Z., Li, F., Liu, B., Liu, Y., Huo, W., Zou, W., Shi, W., 2020. MVP: detecting vulnerabilities using patch-enhanced vulnerability signatures. In: Capkun, S., Roesner, F. (Eds.), 29th USENIX Security Symposium. USENIX Security 2020, August 12-14, 2020, USENIX Association, pp. 1165–1182, URL https://www.usenix.org/conference/usenixsecurity20/presentation/xiao.

Xiaomeng, W., Tao, Z., Runpu, W., Wei, X., Changyu, H., 2018. CPGVA: Code property graph based vulnerability analysis by deep learning. In: 2018 10th International Conference on Advanced Infocomm Technology. ICAIT, IEEE, Stockholm, Sweden, pp. 184–188. http://dx.doi.org/10.1109/ICAIT.2018.8686548, URL https://ieeexplore.ieee.org/document/8686548/.

Xu, X., Liu, C., Feng, Q., Yin, H., Song, L., Song, D., 2017. Neural network-based graph embedding for cross-platform binary code similarity detection. In: Thuraisingham, B., Evans, D., Malkin, T., Xu, D. (Eds.), Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017, ACM, pp. 363–376. http://dx.doi.org/10.1145/3133956.3134018.

Yamaguchi, F., Golde, N., Arp, D., Rieck, K., 2014. Modeling and discovering vulnerabilities with code property graphs. In: 2014 IEEE Symposium on Security and Privacy. IEEE, San Jose, CA, pp. 590–604. http://dx.doi.org/10.1109/SP.2014.44, URL http://ieeexplore.ieee.org/document/6956589/.

Yamaguchi, F., Wressnegger, C., Gascon, H., Rieck, K., 2013. Chucky: exposing missing checks in source code for vulnerability discovery. In: CCS '13: Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security.

Yang, S., Cheng, L., Zeng, Y., Lang, Z., Zhu, H., Shi, Z., 2021. Asteria: Deep learning-based AST-encoding for cross-platform binary code similarity detection. In: 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks. DSN, IEEE, Taipei, Taiwan, pp. 224–236. http://dx.doi.org/10.1109/DSN48987.2021.00036, URL https://ieeexplore.ieee.org/document/9505086/.

Yang, S., Dong, C., Xiao, Y., Cheng, Y., Shi, Z., Li, Z., Sun, L., 2024. Asteria-pro: Enhancing deep learning-based binary code similarity detection by incorporating domain knowledge. ACM Trans. Softw. Eng. Methodol. 33 (1), 1–40. http://dx.doi.org/10.1145/3604611, URL https://dl.acm.org/doi/10.1145/3604611.

Zhang, J., Wang, X., Zhang, H., Sun, H., Wang, K., Liu, X., 2019. A novel neural source code representation based on abstract syntax tree. In: 2019 IEEE/ACM 41st International Conference on Software Engineering. ICSE, IEEE, Montreal, QC, Canada, pp. 783–794. http://dx.doi.org/10.1109/ICSE.2019.00086, URL https://ieeexplore.ieee.org/document/8812062/.

Zheng, W., Jiang, Y., Su, X., 2021a. Vu1SPG: Vulnerability detection based on slice property graph representation learning. In: 2021 IEEE 32nd International Symposium on Software Reliability Engineering. ISSRE, IEEE, Wuhan, China, pp. 457–467. http://dx.doi.org/10.1109/ISSRE52982.2021.00054, URL https://ieeexplore.ieee.org/document/9700294/.

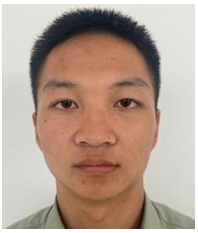
Zheng, Y., Pujar, S., Lewis, B., Buratti, L., Epstein, E., Yang, B., Laredo, J., Morari, A., Su, Z., 2021b. D2A: A dataset built for AI-based vulnerability detection methods using differential analysis. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice. ICSE-SEIP, IEEE, Madrid, ES, pp. 111–120. http://dx.doi.org/10.1109/ICSE-SEIP52600.2021.00020, URL https://ieeexplore.ieee.org/document/9402126/.

Zhiquan, T., Qiao, H., Yupeng, H., Wenxin, K., Jiongyi, C., 2022. SEVulDet: A semantics-enhanced learnable vulnerability detector. In: 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks. DSN 2022.

Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. arXiv arXiv:1909.03496 [cs, stat].

Zou, D., Hu, Y., Li, W., Wu, Y., Zhao, H., Jin, H., 2024. MVulPreter: A multi-granularity vulnerability detection system with interpretations. IEEE Trans. Dependable Secure Comput. 1–12. http://dx.doi.org/10.1109/TDSC.2022.3199769, URL https://ieeexplore.ieee.org/document/9864301/.

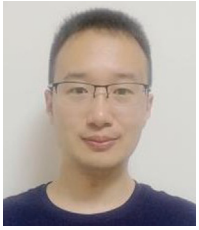
Zou, D., Wang, S., Xu, S., Li, Z., Jin, H., 2019. μVulDeePecker: A deep learning-based system for multiclass vulnerability detection. IEEE Trans. Dependable Secure Comput. 1. http://dx.doi.org/10.1109/TDSC.2019.2942930, URL https://ieeexplore.ieee.org/document/8846081/.



Chen Liang was born in 2000 and received the B.S. degree in software engineering from National University of Defence Technology, Changsha, China in 2022. Now he is studying at Information Engineering University, Zhengzhou, China. His research interests include Code similarity analysis, vulnerability detection.

Qiang Wei is currently a professor at the National Digital Switching System Engineering and Technological Research Center, Zhengzhou, China. His current research interests include network security and vulnerability discovery.

Jiang Du was born in 1990 and pursued undergraduate studies at Xidian University from 2008 to 2012, earning a Bachelor's degree in Engineering. Continuing the academic journey, the author furthered their education at the Information Engineering University, where they completed a Master's degree in Engineering in 2015.



Yisen Wang was born in 1990. He received the B.A. degree from Tianjin University, in 2012,and the M.S. degree in computer science and technology from Information Engineering University, in 2015. He is currently pursuing the Ph.D. degree in computer science and technology with the State Key Laboratory of Mathematical Engineering and Advanced Computing. His research interests include computer architecture, Internet of Things security, and deep learning.

Zirui Jiang was born in 2001 and pursed undergraduate studies at Information Engineering University from 2019 to 2023, earning a Bachelor's degree in Engineering. To continue his academic journey, the author furthered his education at the Information Engineering University and is currently pursuing a master's degree in graduate school.